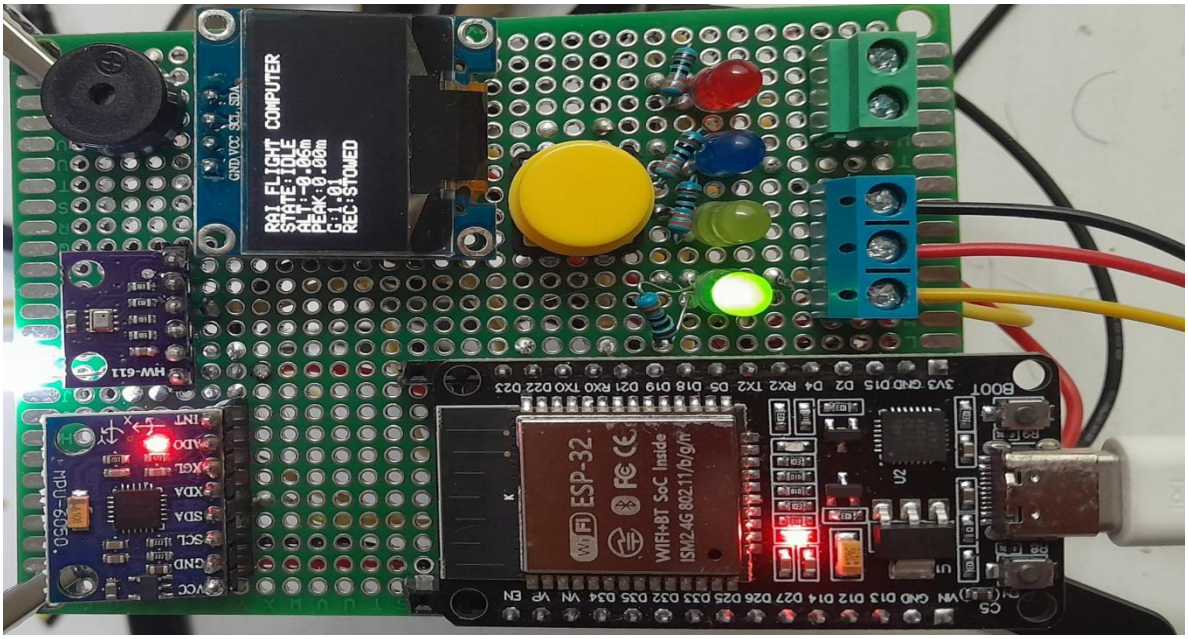
Flight Computer V1

Functional Prototype & Bench Validation Report



Organization
Roshan Aerospace Industries

Project
Flight Computer V1

Document ID
RAI-FC-2026-001

Firmware
FC_V1_v1.6.5_ButtonIntegrated.ino

Project Status
Functional Prototype — Bench Validated

Validation Result
5/5 Full Functional Bench Tests Passed

Year
2026

Document Control

Document Information

Field	Information
Document ID	RAI-FC-2026-001
Document Title	Flight Computer V1 — Functional Prototype & Bench Validation Report
Project	Flight Computer V1
Hardware Revision	V1
Firmware Revision	V1.6.5
Firmware File	FC_V1_v1.6.5_ButtonIntegrated.ino
Document Revision	1.0
Document Status	Final Prototype Documentation
Validation Status	Bench Validated
Validation Result	5/5 Functional Validation Cycles Passed
Organization	Roshan Aerospace Industries
Year	2026

Revision History

Revision	Development Stage / Change	Status
V1.0	Initial component-level development and ESP32 test platform	Superseded
V1.1	Individual sensor and peripheral integration	Superseded
V1.2	Combined sensor acquisition and basic flight-state development	Superseded
V1.3	Flight-event detection and recovery-control development	Superseded
V1.4	State-machine and bench-test logic refinement	Superseded
V1.5	False-trigger mitigation and flight-event validation improvements	Superseded
V1.6.x	Perfboard hardware integration, status indication and final logic refinement	Superseded
V1.6.5	Physical button integration; final validated FC_V1 firmware	FROZEN / FINAL

Document Classification: Engineering Project Documentation
Prototype Classification: Experimental / Bench-Validated Prototype
Flight Qualification: Not flight-qualified

1. Executive Summary

Flight Computer V1 (FC_V1) is an experimental embedded flight-computer prototype developed by Roshan Aerospace Industries to investigate fundamental avionics, sensor-integration, flight-state detection and recovery-control concepts.

The system is built around an ESP32 microcontroller and integrates a BMP280 barometric pressure sensor, MPU6050 inertial measurement unit, OLED display, servo actuator, audible buzzer, status LEDs and a physical control button. The final hardware was assembled on perfboard following individual component testing and breadboard-based system integration.

FC_V1 implements an autonomous sequential state machine:

BOOT → SELF TEST → CALIBRATION → IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

Barometric altitude and acceleration measurements are processed to distinguish flight events. Launch detection uses acceleration together with altitude-change confirmation to reduce false triggering. During ascent, the system tracks peak altitude and identifies apogee after a confirmed altitude decrease. A servo output is subsequently commanded to represent recovery-system deployment. During descent, altitude stability and acceleration magnitude are evaluated to determine landing, after which an audible recovery beacon is activated.

The final firmware, FC_V1_v1.6.5_ButtonIntegrated.ino, also provides physical ARM, DISARM and RESET control, allowing the complete operational sequence to be executed without Serial Monitor commands.

Following hardware and software integration, the completed prototype underwent five consecutive full functional validation cycles. All five tests successfully completed the intended operational sequence, resulting in a **5/5 validation pass**.

FC_V1 is classified as a **functional, bench-validated experimental prototype**. The project demonstrates integrated embedded-system functionality and flight-event logic under controlled bench and hand-motion testing. It has not undergone real-flight, vibration, environmental or high-dynamic qualification testing and therefore is **not considered flight-qualified hardware**.

2. Project Objectives

The primary objective of Flight Computer V1 was to design, construct and validate an experimental embedded avionics platform capable of processing sensor measurements and autonomously identifying major events within a simplified model-rocket flight sequence.

Development focused on practical system integration rather than flight qualification. Individual sensors and peripherals were first tested independently before being combined into a complete perfboard-based prototype.

2.1 Primary Technical Objectives

The project was developed to demonstrate:

- integration of multiple sensors and peripherals with an ESP32 microcontroller;
- barometric relative-altitude measurement using the BMP280;
- three-axis acceleration measurement using the MPU6050;
- automatic startup self-test and sensor calibration;
- implementation of a deterministic flight-state machine;
- launch detection using acceleration and altitude-change confirmation;
- ascent monitoring and peak-altitude tracking;
- apogee detection based on confirmed altitude decrease;
- servo-controlled recovery-system output;
- descent and stationary-landing detection;
- visual system-state indication using LEDs and an OLED display;
- audible event and post-landing recovery indication;
- physical ARM, DISARM and RESET control;
- mitigation of false state transitions caused by normal sensor noise; and
- integration of the complete electronics system onto a soldered perfboard prototype.

2.2 Development Objectives

In addition to the final system functionality, FC_V1 was intended to establish practical experience in:

Embedded electronics: GPIO configuration, I²C communication, power and ground distribution, soldering and perfboard construction.

Sensor processing: calibration, filtering, threshold selection, sensor validation and interpretation of barometric and inertial measurements.

Embedded software: state-machine architecture, event detection, actuator control, debouncing and non-blocking timing.

Verification: systematic component testing, integrated bench testing, fault isolation and repeatable end-to-end functional validation.

2.3 Functional Requirements

The functional requirements defined for FC_V1 establish the minimum behaviours required for the prototype to satisfy its intended bench-validation objectives. Each requirement was evaluated during component-level, integrated and full-sequence testing.

ID	Functional Requirement	Verification
FCV1-REQ-001	The system shall initialize the ESP32 and connected peripherals during startup.	Bench Test
FCV1-REQ-002	The system shall perform startup sensor checks before entering normal operation.	Bench Test
FCV1-REQ-003	The system shall establish a relative altitude reference during initialization.	Bench Test
FCV1-REQ-004	The system shall acquire barometric altitude data from the BMP280.	Component / Integrated Test
FCV1-REQ-005	The system shall acquire acceleration data from the MPU6050.	Component / Integrated Test
FCV1-REQ-006	The system shall provide physical ARM, DISARM and RESET control.	Functional Test
FCV1-REQ-007	The system shall detect a launch event using defined acceleration and altitude-change criteria.	Simulated Flight Test
FCV1-REQ-008	The system shall track relative altitude and peak altitude during simulated ascent.	Simulated Flight Test
FCV1-REQ-009	The system shall identify apogee following a confirmed decrease from recorded peak altitude.	Simulated Flight Test
FCV1-REQ-010	The system shall command the recovery servo following confirmed apogee detection.	Functional Test
FCV1-REQ-011	The system shall identify the descent phase following recovery deployment.	Simulated Flight Test
FCV1-REQ-012	The system shall determine landing using altitude stability and acceleration	Simulated Flight Test

	criteria.	
FCV1-REQ-013	The system shall activate an audible recovery indication following confirmed landing.	Functional Test
FCV1-REQ-014	The system shall display relevant system and flight-state information on the OLED.	Visual Inspection
FCV1-REQ-015	The system shall provide visual state indication using dedicated LEDs.	Visual Inspection
FCV1-REQ-016	The system shall prevent normal ground-level disturbances from causing unintended flight-state transitions within the validated test conditions.	Bench Test
FCV1-REQ-017	The integrated prototype shall complete the intended operational sequence repeatedly without manual firmware intervention.	Full Functional Validation

2.4 Validation Criteria

FC_V1 was considered functionally validated when the completed hardware and frozen firmware successfully demonstrated the intended operational sequence during **five consecutive full bench-validation cycles** without unintended state transitions or functional failure.

The resulting validation criterion was:

Required: 5 consecutive successful functional cycles
Achieved: 5 consecutive successful functional cycles
Result: PASS

3. System Architecture

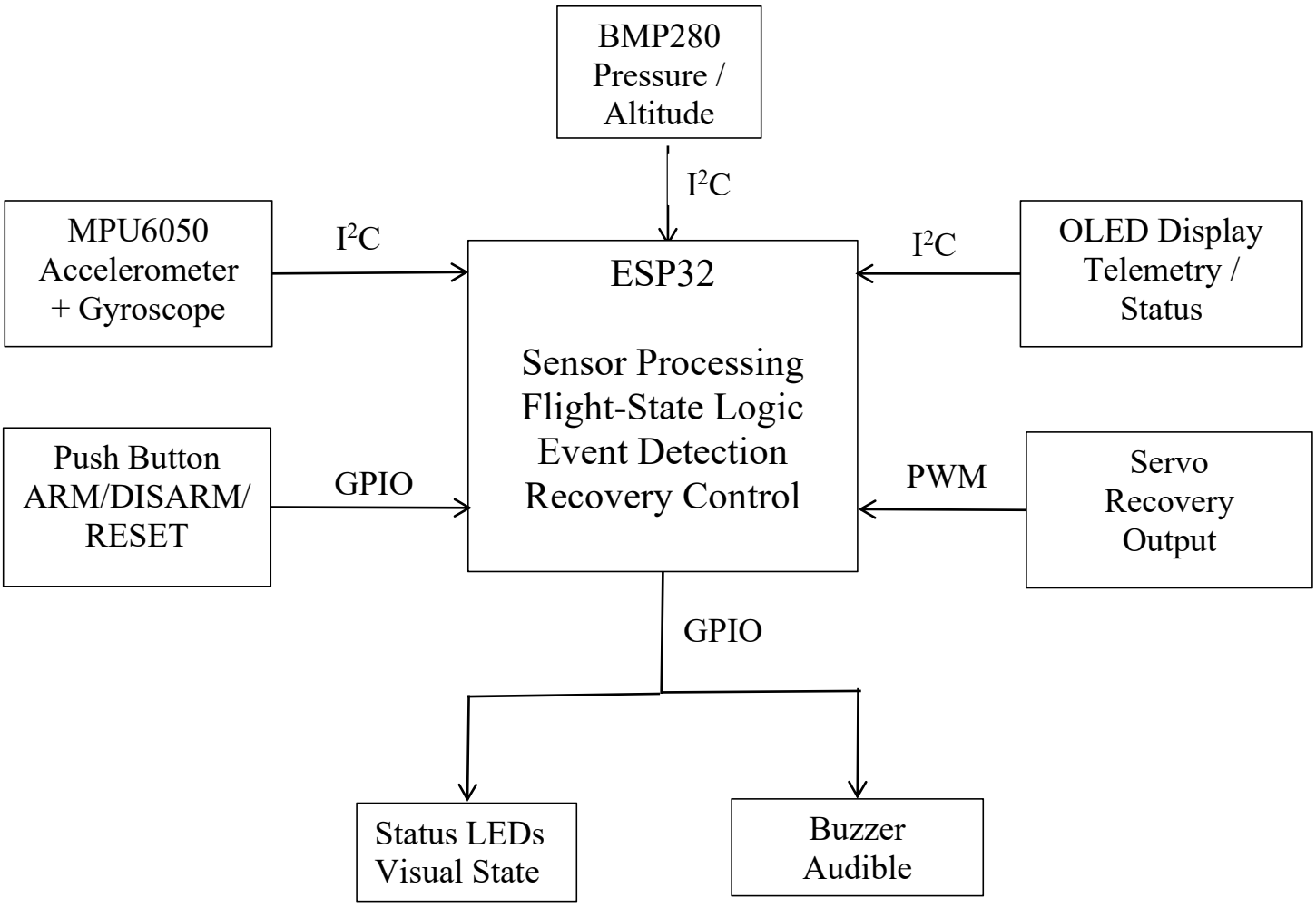
Flight Computer V1 is based on a centralized embedded architecture in which an ESP32 serves as the primary processing and control unit. Sensor measurements are acquired from the BMP280 barometric pressure sensor and MPU6050 inertial measurement unit, while system information and operational states are communicated through an OLED display, dedicated status LEDs and an audible buzzer.

The ESP32 executes the flight-state logic, evaluates sensor measurements, determines state transitions and commands the recovery servo when the defined deployment conditions are satisfied. A physical push button provides local ARM, DISARM and RESET control.

The BMP280, MPU6050 and OLED communicate with the ESP32 through the I²C bus. Other peripherals—including the servo, buzzer, status LEDs and control button—interface through dedicated GPIO connections.

The architecture was intentionally kept simple and modular so that individual subsystems could be independently tested, replaced and debugged during prototype development.

3.1 System Functional Architecture



*[**Figure 3-1**. Functional architecture of Flight Computer V1.]*

The functional architecture separates FC_V1 into four primary groups:

Sensing: BMP280 and MPU6050 provide environmental and inertial measurements used by the flight-event logic.

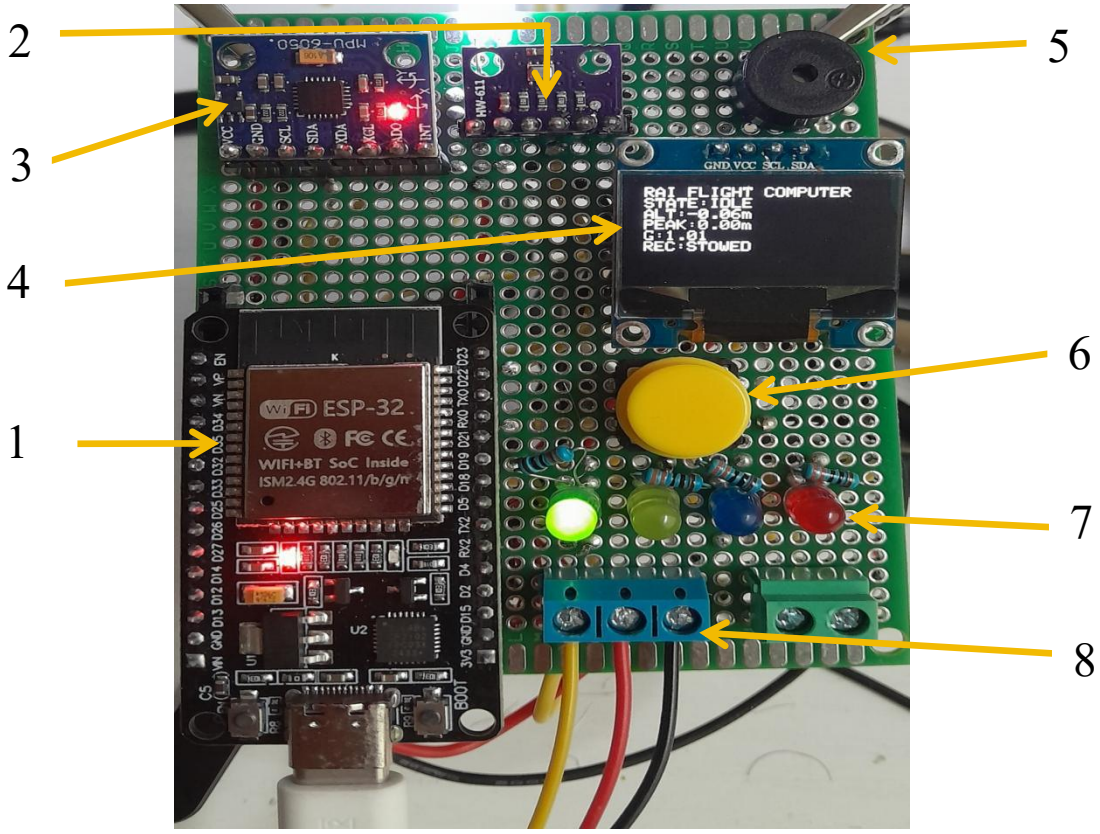
Processing: The ESP32 performs sensor acquisition, calibration, filtering, state evaluation and output control.

Human-machine interface: The OLED, LEDs and push button provide system information, visual state indication and physical user control.

Actuation and recovery indication: The servo represents the recovery deployment actuator, while the buzzer provides audible indication following landing and during defined system events.

3.2 Hardware Architecture

Subsystem	Component	Primary Function	Interface
Processing	ESP32 DevKit	Main processor and flight logic	—
Barometric sensing	BMP280	Pressure and relative-altitude measurement	I²C
Inertial sensing	MPU6050	Acceleration and rotational-rate measurement	I²C
Display	0.96" OLED	Telemetry and system-state display	I²C
Recovery output	Servo	Simulated recovery deployment actuation	PWM/GPIO
Audible output	Buzzer	Event/recovery indication	GPIO
Visual indication	Status LEDs	Flight/system-state indication	GPIO
User input	Push button	ARM, DISARM and RESET	GPIO



1 — ESP32
2 — BMP280
3 — MPU6050
4 — OLED
5 — Buzzer
6 — Control Button
7 — Status LEDs
8 — Servo/External Connections

*[**Figure 3-2.** Completed FC_V1 perfboard prototype showing major hardware subsystems.]*

4. Hardware Design and Implementation

FC_V1 was developed using a modular hardware architecture centred around an ESP32 development board. During early development, individual sensors and peripherals were evaluated independently on a breadboard before being combined into the complete system.

Following successful component-level and integrated breadboard testing, the final prototype was transferred to a soldered perfboard implementation. Female header connectors were used for the ESP32, BMP280 and MPU6050 modules to permit removal and replacement during development and troubleshooting.

Electrical interconnections were implemented using insulated solid-core hookup wire. Common power, ground and signal connections were distributed across the perfboard as required by the individual subsystems.

The final arrangement was selected to maintain access to the ESP32 USB interface, provide visibility of the OLED and status indicators, and retain accessible external connections for the recovery servo and associated peripherals.

4.1 Central Processing Unit — ESP32

The ESP32 development board serves as the primary processing unit of FC_V1. It is responsible for sensor acquisition, calibration, flight-state evaluation, event detection, display management and peripheral control.

The ESP32 was selected for the prototype because it provides sufficient GPIO resources, I²C communication capability, PWM-capable outputs and processing capacity for the required embedded functions.

Within FC_V1, the ESP32 performs the following primary tasks:

- acquisition of BMP280 and MPU6050 sensor measurements;
- establishment and processing of relative altitude;
- evaluation of acceleration magnitude;
- execution of the flight-state machine;
- launch, apogee, descent and landing detection;
- recovery-servo control;
- OLED telemetry generation;
- LED state indication;
- buzzer control; and
- physical-button processing.

4.2 Sensor Subsystem

FC_V1 incorporates two primary sensing devices: the BMP280 barometric pressure sensor and the MPU6050 inertial measurement unit.

4.2.1 BMP280 Barometric Sensor

The BMP280 provides atmospheric-pressure measurements used by FC_V1 to derive barometric altitude. Rather than relying on absolute altitude as the primary flight variable, the system establishes a ground-reference altitude during initialization and subsequently evaluates altitude relative to that reference.

Within the current firmware, BMP280 measurements contribute to:

- relative-altitude determination;
- altitude-change confirmation during launch detection;
- ascent monitoring;
- peak-altitude tracking;
- apogee detection;
- descent monitoring; and
- altitude-stability evaluation during landing detection.

The BMP280 communicates with the ESP32 through the shared I²C bus.

4.2.2 MPU6050 Inertial Measurement Unit

The MPU6050 combines a three-axis accelerometer and three-axis gyroscope within a single module. FC_V1 primarily uses acceleration measurements for flight-event logic.

Acceleration data are used to calculate the resultant acceleration magnitude and provide an additional dynamic measurement for launch and landing evaluation.

The MPU6050 communicates with the ESP32 through the same I²C bus used by the BMP280 and OLED.

4.3 Human-Machine Interface

FC_V1 incorporates several interface components to provide direct information and control during testing.

OLED Display

The OLED provides real-time information including system state, relative altitude, recorded peak altitude, acceleration magnitude and recovery status.

Status LEDs

Dedicated LEDs provide immediate visual indication of selected system and flight states. This allows the operational sequence to be observed without relying exclusively on the Serial Monitor or OLED.

Control Button

A physical push button provides local interaction with the flight computer. In the frozen V1.6.5 firmware, the button supports ARM, DISARM and RESET functionality according to the implemented press-duration logic.

Audible Buzzer

The buzzer provides audible system indication and acts as a recovery beacon following confirmed landing.

4.4 Recovery Actuation

A servo output is incorporated into FC_V1 to represent mechanical actuation of a recovery deployment mechanism.

Following confirmed apogee detection, the firmware commands the servo to its deployment position and records the recovery state as deployed. The servo implementation allows the complete detection-to-actuation sequence to be evaluated during bench testing.

The servo output represents a simulated recovery-system actuator. FC_V1 has not been validated with a physical parachute deployment mechanism.

4.5 Electrical Interface and GPIO Allocation

Function	ESP32 Connection	Function
I²C SDA	GPIO 21	Shared data line for BMP280, MPU6050 and OLED
I²C SCL	GPIO 22	Shared clock line for BMP280, MPU6050 and OLED
Servo signal	GPIO 18	Recovery-actuator command
Buzzer	GPIO 25	Audible event / recovery indication
Green LED	GPIO 26	System-state indication
Yellow LED	GPIO 27	System-state indication
Blue LED	GPIO 32	Flight-state indication
Red LED	GPIO 33	Recovery / warning indication
Push button	GPIO 14	ARM, DISARM and RESET input

The I²C devices share the same SDA and SCL lines, while the servo, buzzer, status LEDs and push button use dedicated GPIO connections.

For the push button, the firmware configures GPIO14 using the ESP32 internal pull-up resistor through INPUT_PULLUP, meaning the input is normally HIGH and becomes LOW when the button connects the GPIO to ground.

4.6 Status-Indicator Logic

The status LEDs provide direct visual indication of defined system and flight states. Their implemented behaviour in the frozen FC_V1 firmware is summarized below.

According to the frozen code:

Flight State	**LED Behaviour**
BOOT / SELF TEST / CALIBRATION	Yellow blinking
IDLE	Green ON
ARMED	Yellow ON
ASCENT	Blue ON
APOGEE	Blue + Red ON
DESCENT	Blue blinking
LANDED	Green ON + Red blinking
FAULT	Red ON

4.7 User-Input Behaviour

The firmware uses:

- **40 ms debounce**
- **2,000 ms long-press threshold**

The functional behaviour is:

Short press in IDLE → ARM
Short press in ARMED → DISARM
Long press ≈ 2 s → RESET to IDLE

The short- and long-press logic is explicitly implemented in the button handler.

ESP32 GPIO18 ── Servo
ESP32 GPIO25 ── Buzzer
ESP32 GPIO26 ── Green LED
ESP32 GPIO27 ── Yellow LED
ESP32 GPIO32 ── Blue LED
ESP32 GPIO33 ── Red LED
ESP32 GPIO14 ── Push Button ── GND

3.3V ── BMP280 / MPU6050 / OLED
GND ── Common ground
Servo power ── separate appropriate supply rail
Servo ground ── common ground

5. Firmware Architecture and Flight-State Logic

The FC_V1 firmware was developed as a state-based embedded control system rather than as a continuously executing collection of independent sensor functions. System behaviour is governed by a defined flight-state machine, with transitions occurring only when the corresponding operational and sensor-based conditions are satisfied.

Firmware revision **FC_V1_v1.6.5_ButtonIntegrated.ino** represents the frozen firmware configuration used during final functional bench validation. The firmware integrates hardware initialization, startup self-test, sensor calibration, sensor acquisition and filtering, flight-event detection, recovery actuation, status indication, user input, telemetry and recovery-beacon functionality.

The primary flight states implemented in the firmware are:

BOOT → SELF_TEST → CALIBRATION → IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

A separate **FAULT** state is provided for unsuccessful startup hardware validation.

These states are directly defined by the firmware's Flight State enumeration.

5.1 Firmware Functional Structure

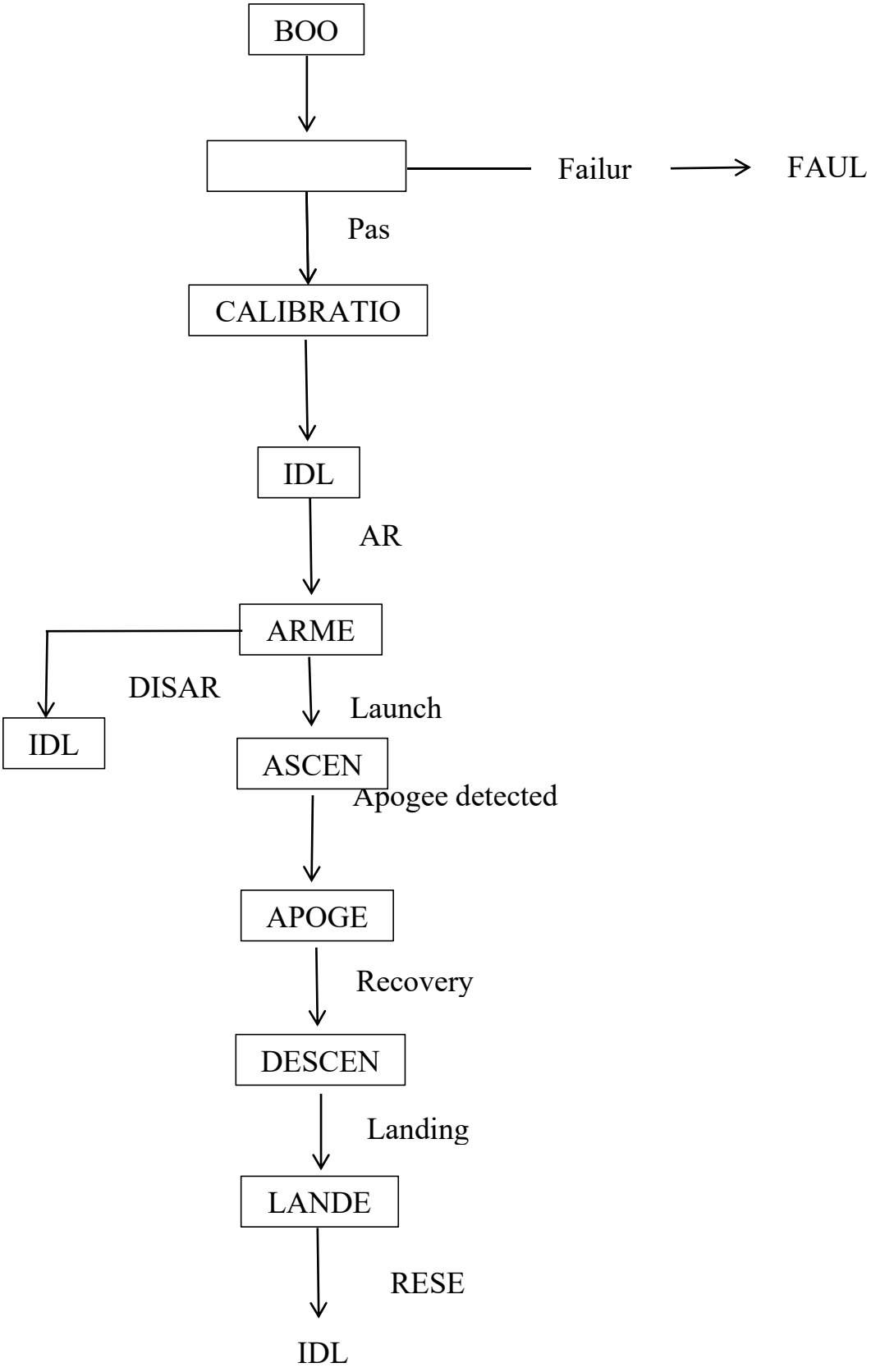
The firmware is divided into functional modules responsible for specific system operations:

Firmware Function	Purpose
Hardware Initialization	Initializes I²C, GPIO, LEDs, servo, buzzer and connected peripherals
Self-Test	Verifies OLED, BMP280 and MPU6050 availability
Calibration	Establishes initial altitude reference and gyroscope offsets
Sensor Acquisition	Reads barometric and inertial measurements
Sensor Filtering	Rejects invalid barometric samples and filters relative altitude

Launch Detection	Detects acceleration event and confirms altitude increase
Apogee Detection	Tracks peak altitude and confirms subsequent altitude decrease
Recovery Control	Commands the recovery servo after confirmed apogee
Landing Detection	Evaluates altitude stability and acceleration magnitude
Status Control	Drives LEDs, OLED and buzzer according to system state
User Input	Processes short- and long-duration button presses
Serial Interface	Provides ARM, DISARM, RESET and STATUS commands
Recovery Beacon	Generates periodic audible indication after landing

That structure is supported by the actual firmware: sensor acquisition is performed on a **100 ms interval**, while button and status handling run continuously in the main loop.

5.2 Flight-State Machine



*[**Figure 5-1.** FC_V1 firmware state-machine architecture showing the nominal operational sequence, manual ARM/DISARM/RESET paths and startup fault path.]*

5.3 Sensor Acquisition and Filtering

FC_V1 acquires environmental and inertial measurements from the BMP280 barometric pressure sensor and MPU6050 inertial measurement unit. Sensor acquisition is performed periodically during operation and provides the measurement inputs required by the flight-event detection algorithms.

The BMP280 provides pressure-derived altitude information. During initialization, the firmware establishes a ground-level reference altitude, allowing subsequent measurements to be expressed as **relative altitude** rather than absolute altitude. This enables the flight-state logic to evaluate vertical displacement relative to the calibrated starting position.

The MPU6050 provides three-axis acceleration measurements. The acceleration components are combined to determine acceleration magnitude, expressed relative to gravitational acceleration:

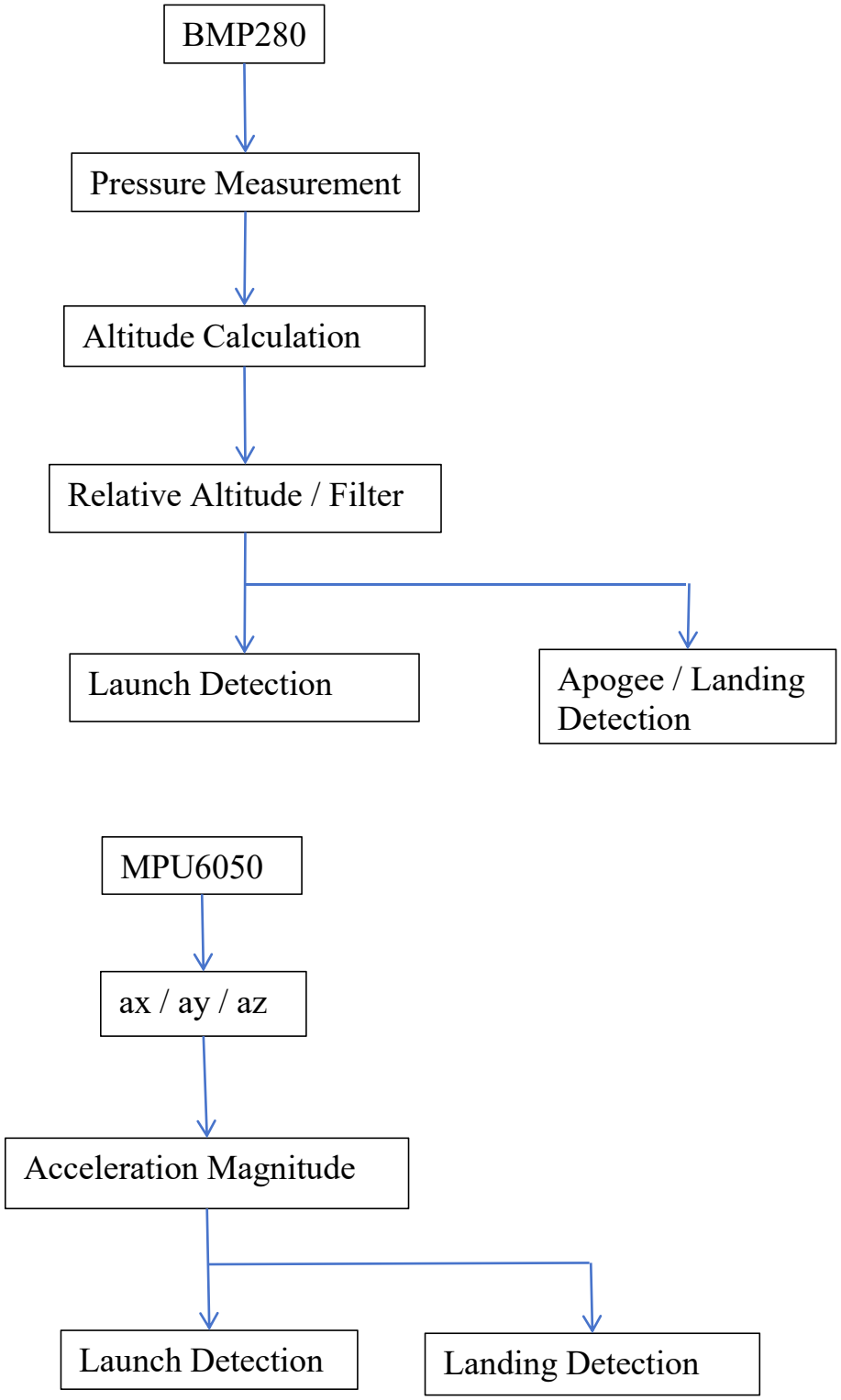
$$\mathbf{G = \sqrt{(ax^2 + ay^2 + az^2)}}$$

where:

ax, ay, az = measured acceleration components expressed in g.

Barometric measurements are subject to filtering and validity checks before being used by the flight-state logic. This reduces the influence of short-duration pressure fluctuations and measurement noise that could otherwise produce false altitude changes.

The processed sensor information is subsequently supplied to the launch, ascent, apogee and landing-detection functions.



*[**Figure 5-2**. Sensor-processing flow from raw BMP280 and MPU6050 measurements to flight-event detection logic.]*

5.4 Launch Detection Logic

Launch detection is enabled only after the system has successfully entered the **ARMED** state. This prevents normal sensor activity during IDLE operation from initiating the flight sequence.

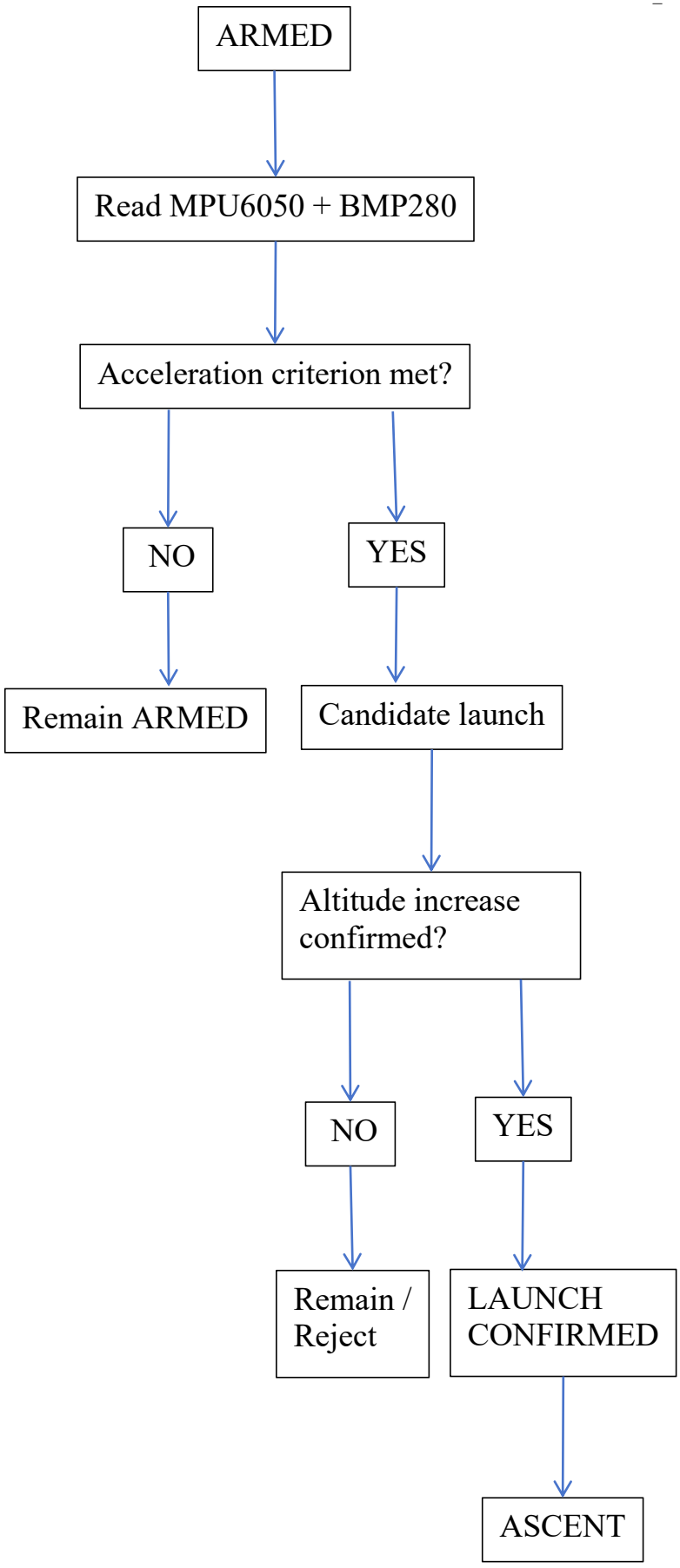
Rather than relying on a single sensor threshold, FC_V1 uses a confirmation-based launch-detection strategy combining inertial and barometric information. An acceleration event is first detected from the MPU6050 measurement. The firmware then evaluates the associated increase in relative altitude before confirming launch.

This approach was introduced to reduce false state transitions caused by handling, vibration, sensor noise or short-duration acceleration disturbances while the system remains near ground level.

Only after the required launch criteria have been satisfied does the state machine transition:

ARMED → ASCENT

Once ASCENT is entered, peak-altitude tracking begins and the system becomes eligible for subsequent apogee detection.



*[**Figure 5-3.** Multi-condition launch-detection logic used to reduce false launch triggering.]*

5.5 Apogee Detection Logic

Following confirmed launch, FC_V1 continuously monitors relative altitude during the **ASCENT** state and maintains a record of the highest observed altitude as the current peak altitude.

Apogee detection is not triggered by a single lower altitude measurement. Instead, the firmware evaluates whether the measured altitude has decreased sufficiently from the recorded peak and requires the decrease condition to remain valid according to the implemented confirmation logic.

This approach reduces the probability that short-duration barometric fluctuations near maximum altitude will be interpreted as a genuine transition from ascent to descent.

Conceptually, the altitude decrease is evaluated as:

$\Delta h = h_{peak} - h_{current}$

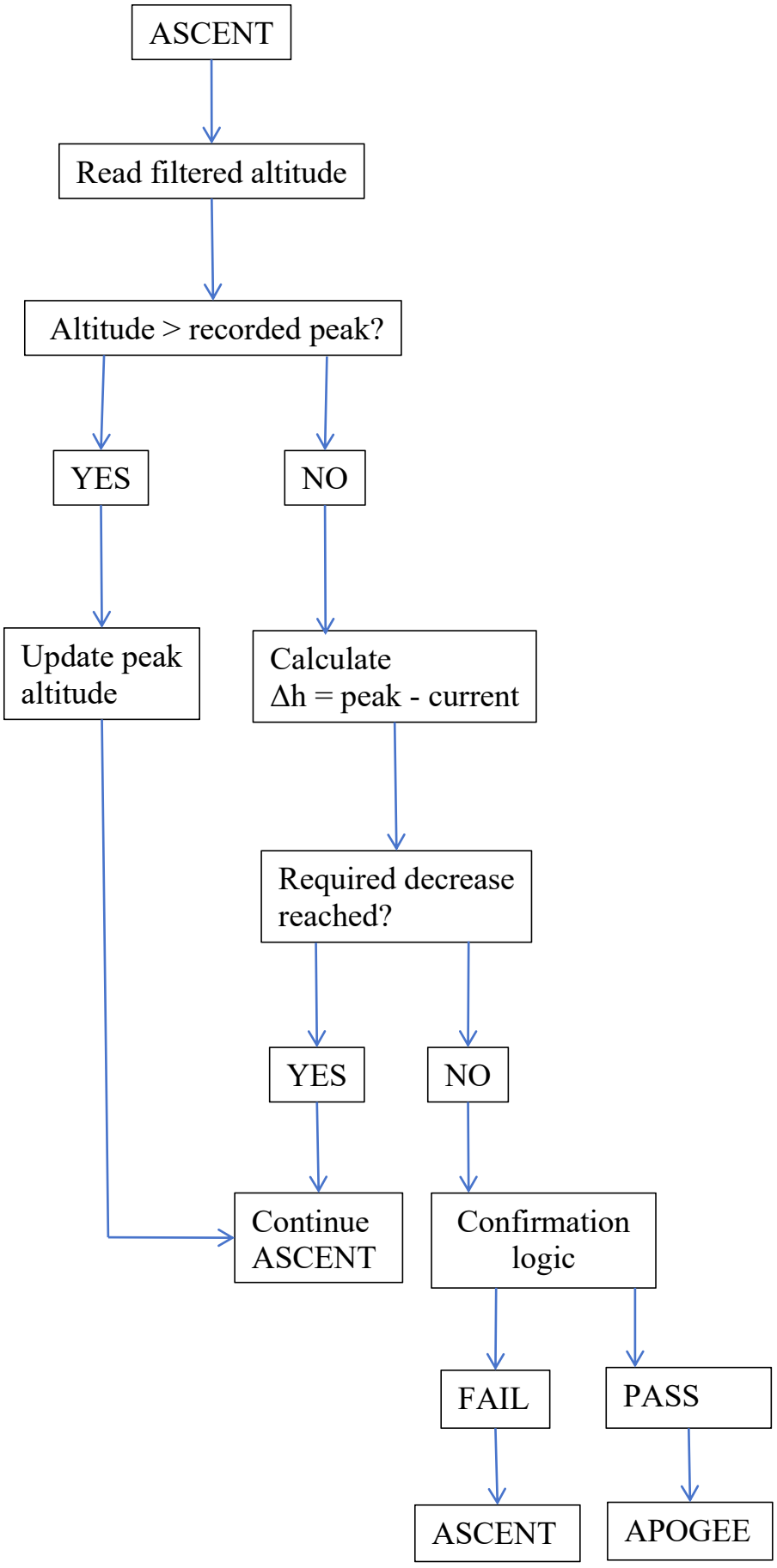
where:

h_{peak} = highest confirmed relative altitude recorded during ascent
$h_{current}$ = current filtered relative altitude
Δh = decrease from recorded peak altitude

When the required decrease and confirmation conditions are satisfied, apogee is declared and the flight-state machine transitions:

ASCENT → APOGEE

Once apogee has been confirmed, the recovery-control sequence becomes eligible for execution. The use of peak tracking combined with confirmation logic provides greater resistance to false apogee detection than a single-sample altitude comparison.



*[**Figure 5-3**. Peak-altitude tracking and confirmation-based apogee-detection logic implemented in FC_V1.]*

5.6 Recovery Actuation Logic

FC_V1 incorporates a servo output to represent the actuation of a recovery mechanism. The servo is not commanded solely from instantaneous altitude data; actuation occurs only after the flight-state logic has confirmed apogee.

Following the transition to **APOGEE**, the firmware executes the defined recovery-control sequence and commands the servo from its stowed position to its deployment position. The recovery state is subsequently recorded internally and communicated through the system's visual status indication.

The functional sequence is:

Confirmed Apogee → APOGEE State → Recovery Command → Servo Actuation → Recovery State Recorded → DESCENT

The servo therefore demonstrates the complete electronic detection-to-actuation chain of the prototype.

The current FC_V1 prototype has **not been validated with an actual parachute or flight-recovery mechanism**. Consequently, servo operation represents recovery-system actuation for functional bench validation rather than a flight-qualified deployment system.

5.7 Landing Detection Logic

Landing detection is evaluated during the **DESCENT** state after recovery actuation has occurred. The objective of the landing-detection logic is to determine when the system has returned to a stationary or near-stationary condition following the simulated flight sequence.

A single altitude measurement is insufficient for reliable landing detection because barometric altitude naturally fluctuates due to sensor noise and environmental pressure variations. Similarly, acceleration magnitude alone may temporarily approach the stationary value during portions of the simulated flight.

FC_V1 therefore evaluates both **altitude stability** and **acceleration magnitude** before confirming landing.
Altitude stability is evaluated from the change in relative altitude over time:

$$\Delta h = |h_{current} - h_{previous}|$$

where:

h_current = current filtered relative altitude
h_previous = previous filtered relative altitude
Δh = measured altitude change

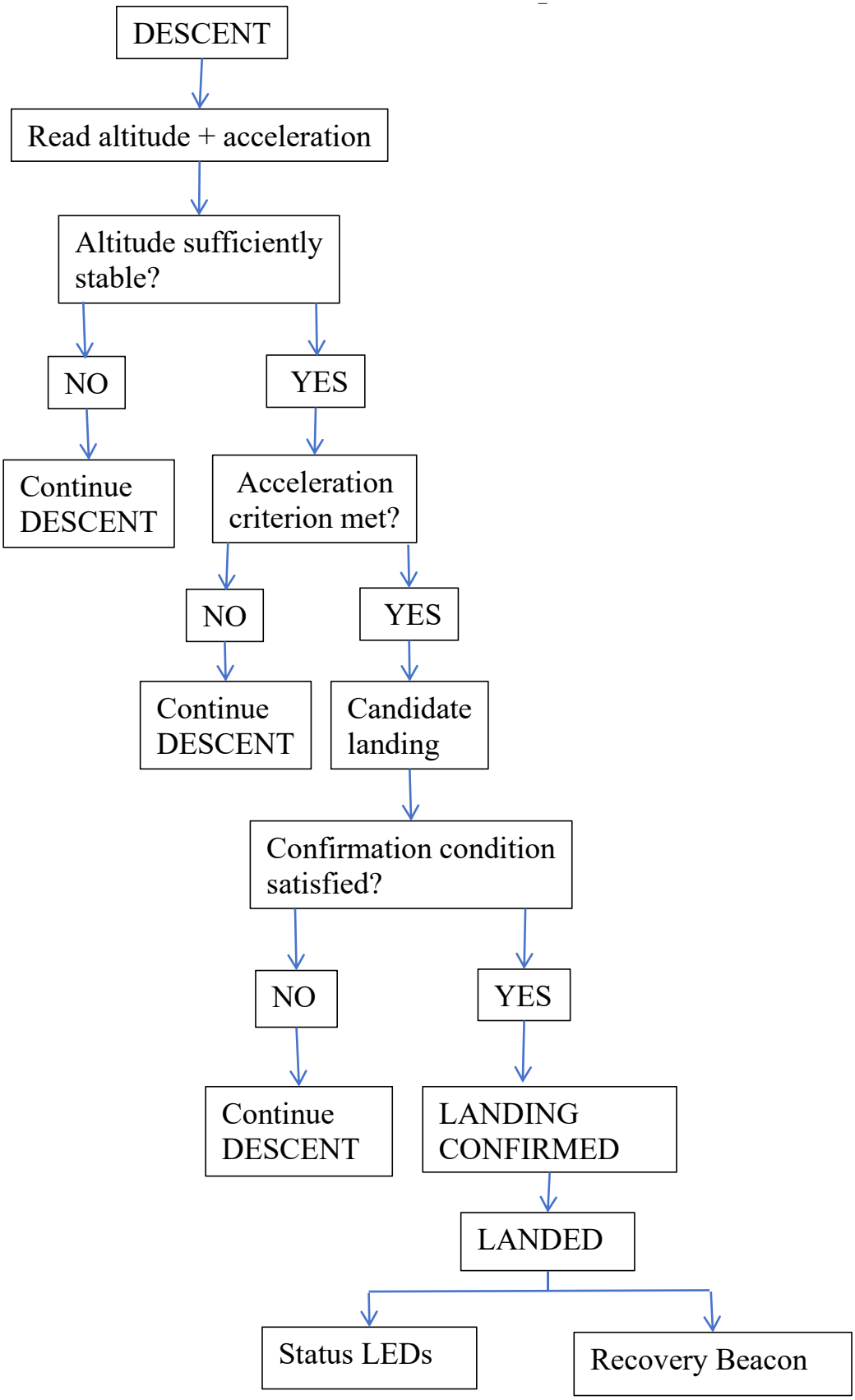
The MPU6050 measurement is simultaneously used to determine whether the acceleration magnitude has returned to approximately the stationary gravitational condition.

A candidate landing condition is accepted only when the required stability conditions persist according to the firmware's confirmation logic. This prevents a short stationary interval or individual sensor fluctuation from immediately producing a landing event.

Following confirmed landing, the state machine transitions:

DESCENT → LANDED

The system then activates its defined post-landing indications, including visual status indication and the audible recovery-beacon functionality.



*[**Figure 5-4**. Multi-condition landing-detection logic combining altitude stability, acceleration and confirmation criteria.]*

5.8 Firmware Parameters and Detection Thresholds

Flight-event detection behaviour is governed by a defined set of firmware parameters and thresholds. These values represent the configuration used in firmware revision **FC_V1_v1.6.5_ButtonIntegrated.ino** during final bench validation.

The parameters were selected and refined through iterative bench testing to obtain repeatable operation under the prototype's simulated flight conditions. They should therefore be interpreted as **bench-validated prototype parameters rather than flight-qualified values**.

Parameter	V1.6.5 Value	Purpose
Sensor acquisition interval	[from frozen code]	Sensor update rate
Launch acceleration threshold	[from frozen code]	Initial launch-event detection
Launch altitude confirmation	[from frozen code]	Reject ground-level acceleration disturbances
Apogee drop threshold	[from frozen code]	Detect decrease from peak altitude
Apogee confirmation	[from frozen code]	Reject short barometric fluctuations
Recovery delay	[from frozen code]	Delay between confirmed apogee and subsequent recovery sequence
Servo stowed position	0°	Recovery actuator stowed state
Servo deployed position	90°	Recovery actuator deployed state
Landing altitude-stability threshold	[from frozen code]	Determine near-stationary altitude
Landing acceleration criterion	[from frozen code]	Determine stationary inertial condition
Landing confirmation period/count	[from frozen code]	Prevent premature landing declaration
Button debounce	40 ms	Reject button contact bounce

Button long-press threshold	2000 ms	RESET command recognition

6. Prototype Development and Hardware Integration

FC_V1 was developed using an incremental prototype-development process in which individual electronic modules were first prepared and verified independently before being combined into a complete system. Initial development was performed on a solderless breadboard to permit rapid wiring changes and subsystem testing. Following successful functional validation, the design was transferred to a hand-wired perfboard assembly.

The resulting prototype integrates the ESP32 processing unit, BMP280 barometric sensor, MPU6050 inertial measurement unit, OLED status display, user push button, visual status indicators, buzzer and external connection terminals on a single physical assembly.

The prototype was developed primarily as a **functional engineering demonstrator** for embedded sensing, flight-state detection and recovery-control logic. It is not intended to represent flight-qualified avionics hardware.

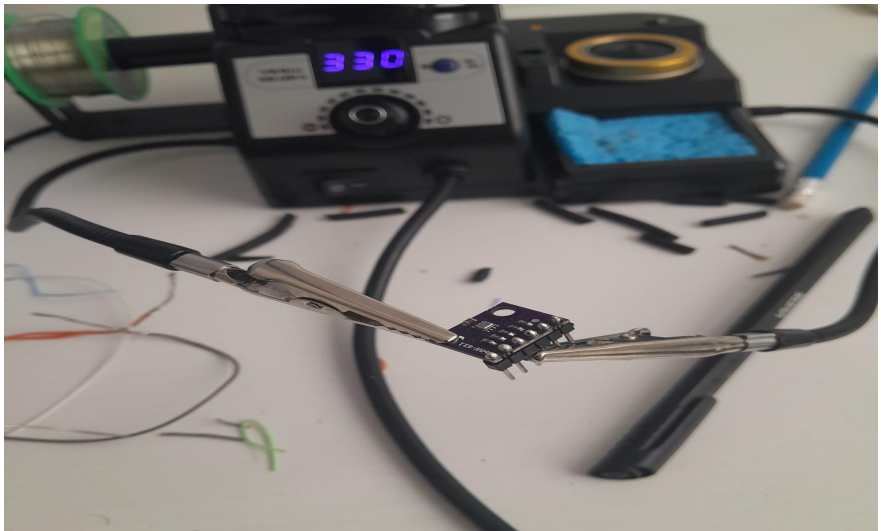
6.1 Component Preparation and Soldering

Development began with the preparation of the individual sensor and interface modules. Header pins were soldered to the BMP280, MPU6050 and OLED modules to provide reliable electrical connections during breadboard testing and subsequent permanent integration.

The soldering process was performed manually using a temperature-controlled soldering station. Particular attention was required around the closely spaced module terminals to prevent solder bridging between adjacent connections.

Following preparation, individual modules were electrically connected to the ESP32 and functionally tested before integration with the remaining hardware. This incremental approach allowed soldering defects, wiring errors and module-specific faults to be identified without introducing the additional complexity of the complete system.

Component preparation therefore established both the physical electrical interfaces and the initial verification baseline for subsequent prototype development.



*[**Figure 6-1**. BMP280 sensor module during header-pin soldering and component preparation for FC_V1 prototype integration.]*

6.2 Breadboard Development and Subsystem Verification

Before construction of the permanent prototype, FC_V1 was assembled and evaluated on a solderless breadboard. This configuration provided a reconfigurable development environment in which wiring, GPIO assignments and individual subsystem behaviour could be modified without permanent electrical connections.

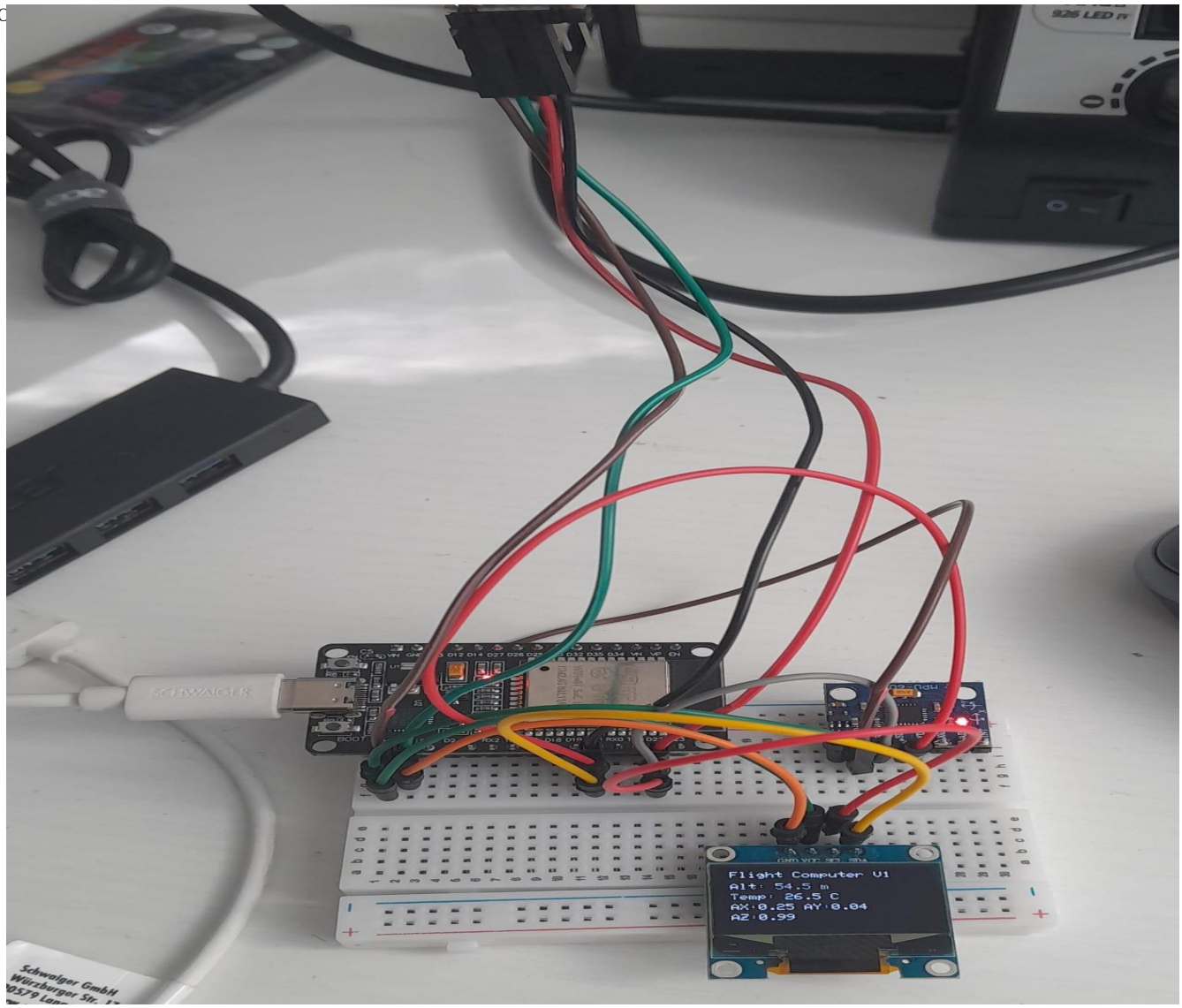
The ESP32 initially served as the central processing platform while the BMP280, MPU6050 and OLED modules were progressively connected through the shared I²C interface. Additional hardware, including the buzzer, servo output and user-interface components, was subsequently introduced after the core sensor system had been verified.

Subsystems were tested individually before combined operation. The principal development sequence consisted of:

1. ESP32 programming and basic execution verification
2. OLED communication and display verification
3. BMP280 pressure and altitude measurement
4. MPU6050 inertial measurement
5. Buzzer output
6. Servo actuation
7. Combined sensor acquisition
8. Flight-state logic and event detection
9. User-input and status-indication integration

The breadboard configuration was particularly useful during firmware development because sensor measurements and state information could be observed through both the OLED interface and serial output while hardware connections remained accessible for modification.

Successful combined operation on the breadboard provided the basis for transferring the system to a more mechanically integrated perfboard configuration.



*[**Figure 6-2**. FC_V1 breadboard development configuration during combined ESP32, BMP280, MPU6050 and OLED subsystem testing.]*

6.3 Perfboard Integration

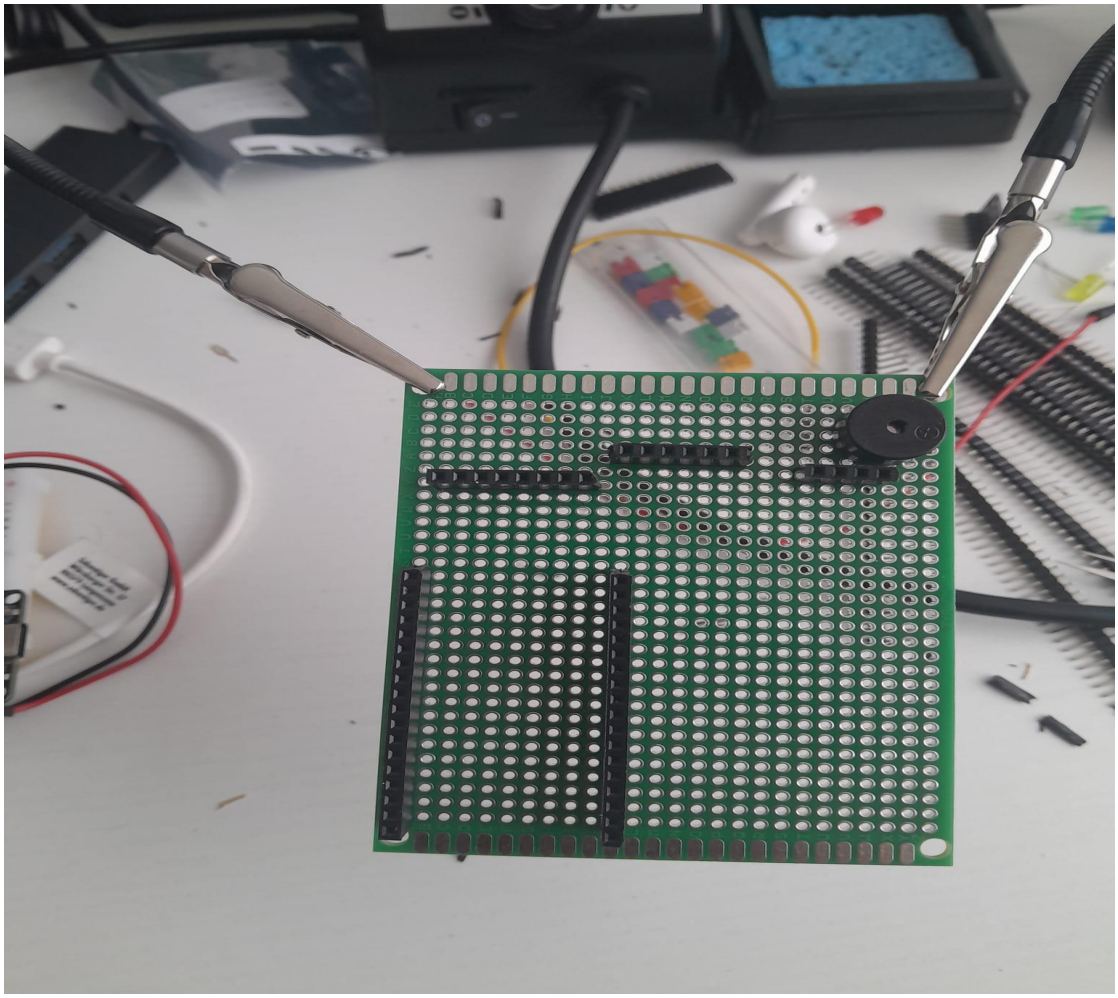
Following successful breadboard validation, the prototype was transferred to a perforated circuit board to create a more compact and mechanically integrated assembly.

Rather than soldering the principal electronic modules permanently to the board, female header sockets were installed for the ESP32 and sensor modules where practical. This allows individual modules to be removed or replaced during development and reduces the risk of damaging the modules during repeated soldering operations.

Component placement was selected to maintain physical separation between the processing unit, sensor modules, display and user-interface hardware while keeping interconnection lengths practical for manual wiring.

The perfboard also provided mounting locations for the status LEDs, current-limiting resistors, buzzer, push button and external screw-terminal connections.

This stage represented the transition from an experimental breadboard system to a self-contained FC_V1 engineering prototype.



***[Figure 6-3.** FC_V1 perfboard during construction, showing the installed female header sockets prior to final component integration and point-to-point wiring.]*

6.4 Completed FC_V1 Hardware Assembly

The completed FC_V1 prototype consolidates the previously verified subsystems onto a single perfboard assembly.

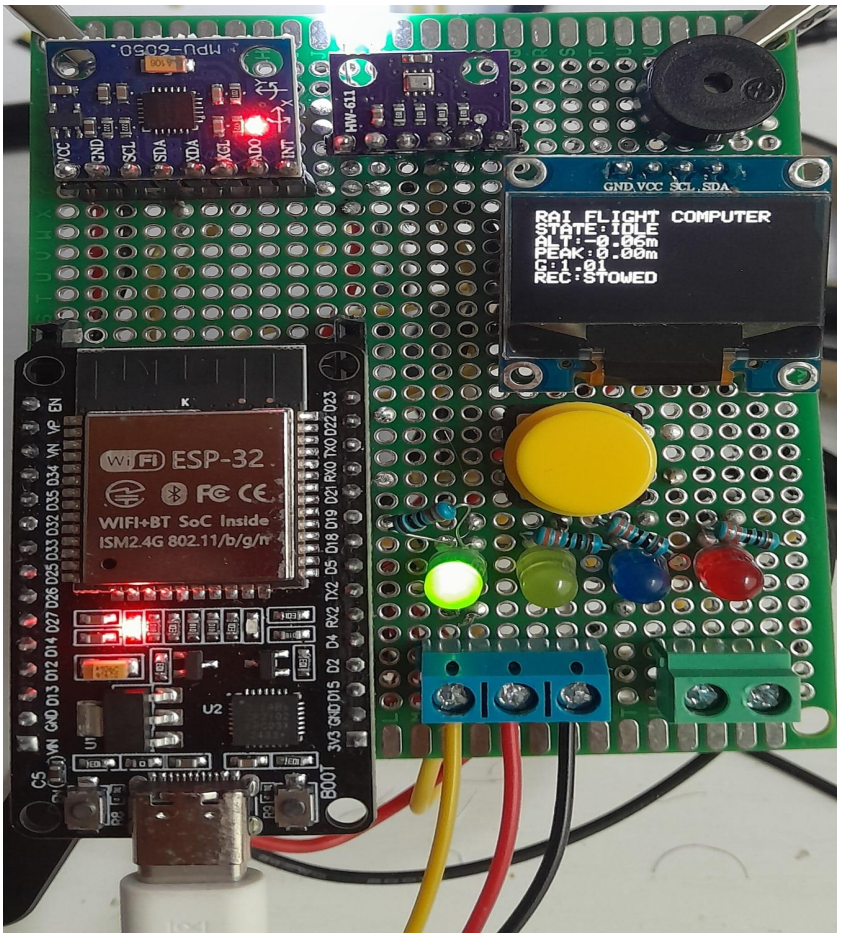
The ESP32 provides the primary processing and firmware-execution platform. The BMP280 and MPU6050 provide barometric and inertial measurements respectively, while the OLED presents real-time system information including flight state, relative altitude, recorded peak altitude, acceleration magnitude and recovery-system status.

A large push button provides direct user input for ARM, DISARM and RESET operations. Four discrete LEDs provide visual state indication, while the buzzer provides audible status and post-landing recovery indication.

Screw-terminal connectors provide external electrical interfaces, including connections associated with the recovery actuator and supporting power or peripheral wiring.

The modular header-based arrangement permits the ESP32 and principal sensor modules to be removed if troubleshooting or replacement becomes necessary.

The completed assembly therefore combines sensing, computation, user interaction, status indication and recovery-control interfaces within a single functional prototype.



*[**Figure 6-4**. Completed FC_V1 perfboard prototype showing the ESP32 processing unit, BMP280 and MPU6050 sensors, OLED interface, user button, status indicators, buzzer and external connection terminals]*

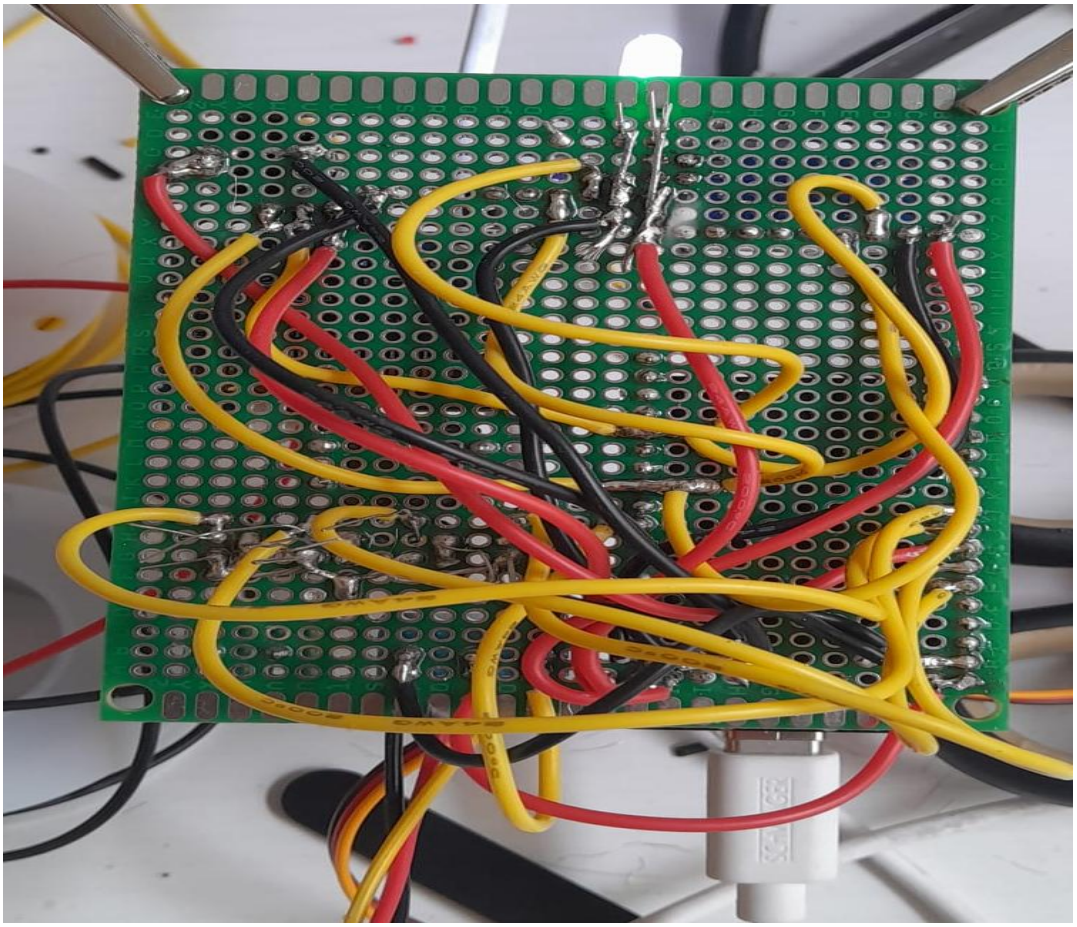
6.5 Point-to-Point Electrical Interconnection

Electrical connections on the perfboard were implemented manually using soldered point-to-point wiring. Power, ground, I²C communication lines, GPIO signals and peripheral connections were routed between the ESP32 and the corresponding hardware components on the underside of the board.

This construction method was suitable for FC_V1 because the primary objective was functional prototype validation rather than production-level packaging or flight-qualified hardware development.

However, manual point-to-point wiring introduces several limitations. The relatively high wiring density reduces maintainability and makes visual inspection and troubleshooting more difficult. Long or unsupported conductors may also be less suitable for environments involving sustained vibration or mechanical loading.

These limitations are accepted for FC_V1 as part of its prototype status. A future hardware revision should reduce wiring complexity through improved board layout and, ultimately, a purpose-designed printed circuit board where justified by the development requirements.



*[**Figure 6-5**. Solder side of the completed FC_V1 prototype showing manually implemented point-to-point power, signal and peripheral interconnections.]*

6.6 Integrated Functional Prototype

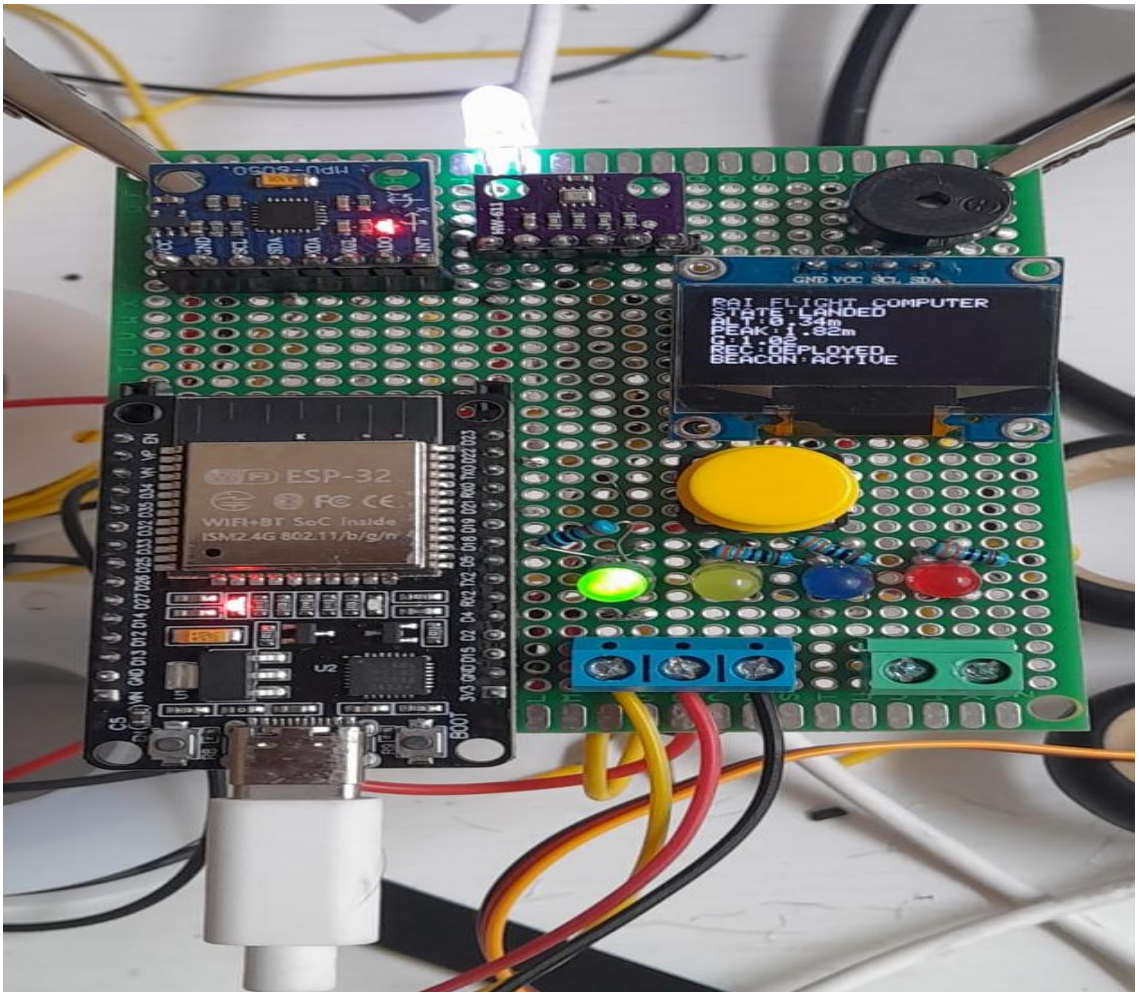
Following hardware integration, the perfboard prototype was operated using the integrated FC_V1 firmware to verify interaction between sensor acquisition, state-machine execution, user input, status indication and recovery-control outputs.

During operation, the OLED provides direct observation of the active flight state and relevant processed sensor quantities. The status LEDs provide an additional visual representation of system condition, while recovery status is represented through the servo-control output and corresponding display indication.

The integrated system was exercised through the implemented flight-state sequence under simulated bench-test conditions. This allowed transitions associated with arming, launch detection, ascent, apogee detection, recovery actuation, descent and landing detection to be evaluated without an actual flight.

Following confirmed landing, the system enters the **LANDED** state and activates the defined post-landing indication and recovery-beacon behaviour. This demonstrates the interaction between flight-event detection and physical output hardware within the complete prototype.

These tests validate the functional integration of FC_V1 under controlled bench conditions. They do **not** constitute environmental qualification, vibration qualification or flight validation.



*[**Figure 6-6**. FC_V1 during integrated bench validation in the LANDED state, showing deployed recovery status and active post-landing recovery-beacon indication.]*

7. Verification and Bench Testing

FC_V1 was subjected to a structured series of bench tests to evaluate the operation of the integrated hardware and firmware under simulated flight-event conditions. The objective of this verification process was to determine whether the prototype could reliably acquire sensor data, execute the implemented flight-state logic, command the recovery output and provide appropriate visual and audible status indications.

Testing was performed under controlled indoor conditions without an actual flight vehicle or parachute deployment system. Flight events were therefore reproduced manually by manipulating the prototype and its sensor inputs to generate representative acceleration and relative-altitude changes.

The verification programme focused on functional behaviour rather than environmental or flight qualification. Particular attention was given to correct state sequencing, rejection of premature state transitions, apogee recognition, recovery actuation, landing detection and post-landing indication.

The principal sequence evaluated during testing was:

IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

Successful completion of this sequence was considered the primary functional acceptance criterion for the integrated FC_V1 prototype.

7.1 Test Objectives and Methodology

The bench-test programme was designed to verify the interaction between the sensing, processing, state-detection and output subsystems after completion of the integrated perfboard prototype.

The principal verification objectives were:

1. Confirm successful initialization of the ESP32 and connected peripherals.
2. Verify stable BMP280 pressure and relative-altitude acquisition.
3. Verify MPU6050 acceleration measurement.
4. Confirm correct IDLE and ARMED behaviour.
5. Verify that launch detection requires the implemented confirmation conditions.
6. Confirm transition from ARMED to ASCENT following a valid simulated launch event.
7. Verify peak-altitude tracking during the simulated ascent.
8. Confirm apogee detection following the required decrease from the recorded peak altitude.
9. Verify recovery-servo actuation only after confirmed apogee.

10. Confirm transition into DESCENT following the recovery sequence.
11. Verify landing detection using the combined altitude-stability and acceleration criteria.
12. Confirm activation of the defined post-landing visual and audible indications.

The OLED display was used as the primary local diagnostic interface during testing. It provided direct observation of the current flight state, relative altitude, recorded peak altitude, acceleration magnitude and recovery-system status.

The status LEDs provided an additional indication of state-machine operation, while the buzzer was used to verify the post-landing recovery-beacon function.

Because the prototype was evaluated manually rather than installed in a flying vehicle, the test results represent **functional bench verification only**.

7.2 Pre-Test Functional Checks

Before each simulated flight sequence, the prototype was powered and allowed to complete initialization. Correct startup behaviour was confirmed through the OLED interface and status indicators.

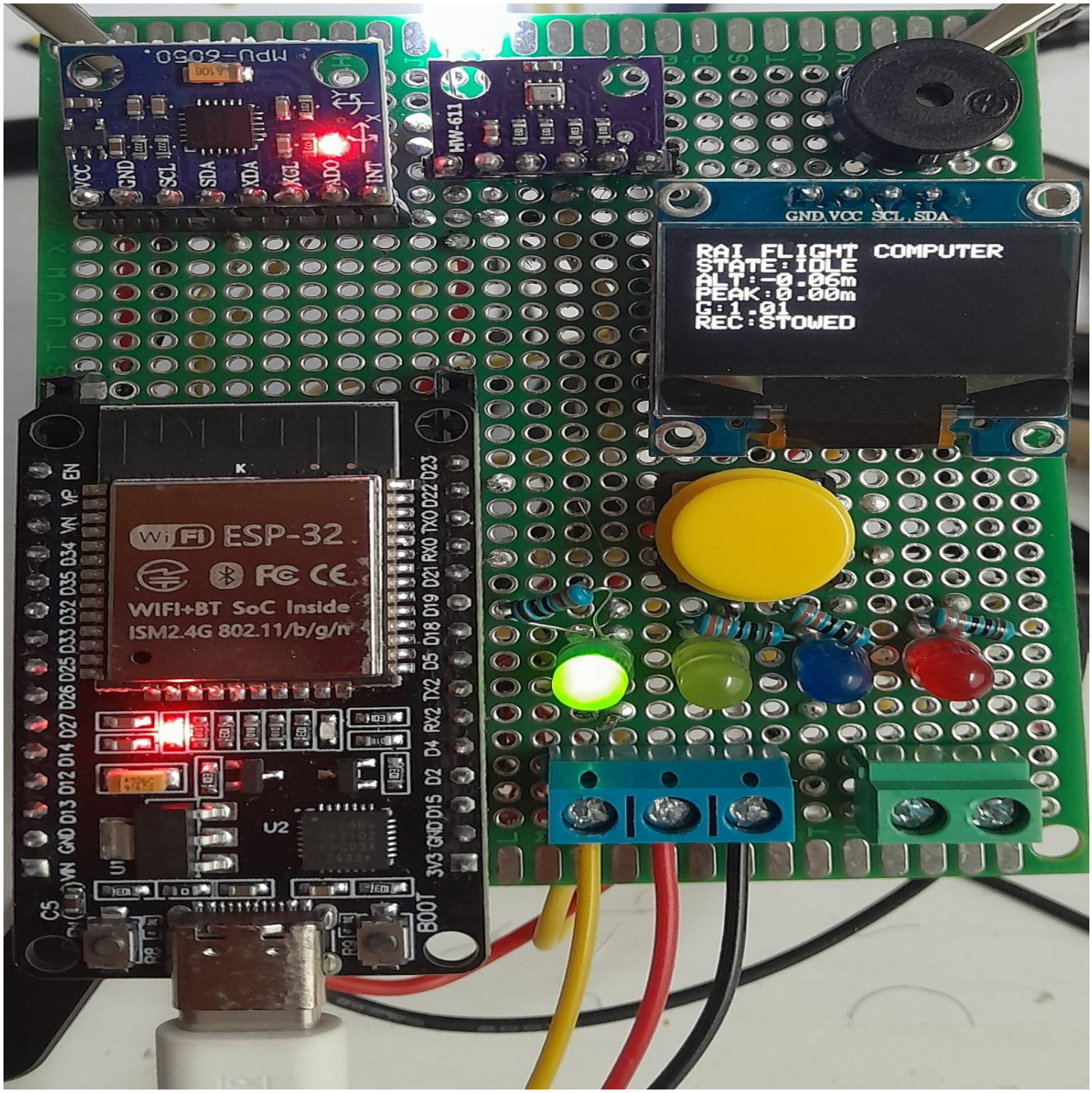
The sensor outputs were observed before arming to confirm that the BMP280 and MPU6050 were providing plausible measurements. Relative altitude was checked for approximately stationary behaviour while the prototype remained at rest, and the acceleration magnitude was verified to remain close to the expected stationary gravitational condition.

The recovery actuator was maintained in its stowed position during initialization and IDLE operation. The flight-event state machine was not permitted to progress into the flight sequence until the system had been deliberately armed through the implemented user interface.

The pre-test procedure therefore provided a basic functional check of:

Power → Firmware Initialization → Sensor Communication → Display Operation → Status Indication → Recovery Stowed → Ready for Arming

Any failure of the required subsystem behaviour would invalidate the subsequent simulated-flight test.



[Figure 7-1. FC_V1 during the pre-test IDLE condition, showing active sensor acquisition and the recovery actuator in the stowed state.]

7.3 Simulated Flight-Test Procedure

Following successful pre-test verification, the system was armed using the prototype's user-input control. Sensor data continued to be acquired while the firmware monitored the launch-detection criteria described in Section 5.

A simulated launch event was then introduced by applying controlled motion to the prototype. The purpose was not to reproduce the exact dynamic environment of a rocket flight, but to provide sensor conditions sufficient to exercise the implemented flight-event detection logic.

Once the acceleration criterion and associated relative-altitude confirmation condition were satisfied, the system transitioned from ARMED to ASCENT.

During the simulated ascent, relative altitude was varied to allow the firmware to establish and continuously update the recorded peak altitude. A subsequent reduction in measured relative altitude was then introduced to exercise the apogee-detection algorithm.

After the configured apogee criteria and confirmation logic were satisfied, the firmware declared APOGEE and initiated the recovery-control sequence. The recovery servo was commanded from its stowed position to its deployed position, after which the system proceeded into DESCENT.

The prototype was subsequently returned to a stable condition. Landing was accepted only after the required altitude-stability, acceleration and confirmation conditions had been satisfied.

The complete simulated test sequence was therefore:

Initialize → IDLE → ARM → Generate Launch Conditions → ASCENT → Establish Peak Altitude → Generate Altitude Decrease → APOGEE → Recovery Actuation → DESCENT → Stabilize Prototype → LANDED

This procedure was repeated to evaluate whether the complete state sequence could be reproduced without unintended transitions.

7.4 Flight-State Transition Verification

The flight-state machine was observed throughout each test using the OLED interface and status LEDs. The principal objective was to verify that the system progressed through the implemented states in the intended order rather than transitioning directly between unrelated flight conditions.

During IDLE operation, sensor measurements remained active while flight-event transitions were inhibited. Following manual arming, the firmware entered ARMED and began evaluating the launch criteria.

A valid simulated launch caused the system to transition:

ARMED → ASCENT

During ASCENT, the recorded peak altitude was updated whenever the measured relative altitude exceeded the previous maximum.

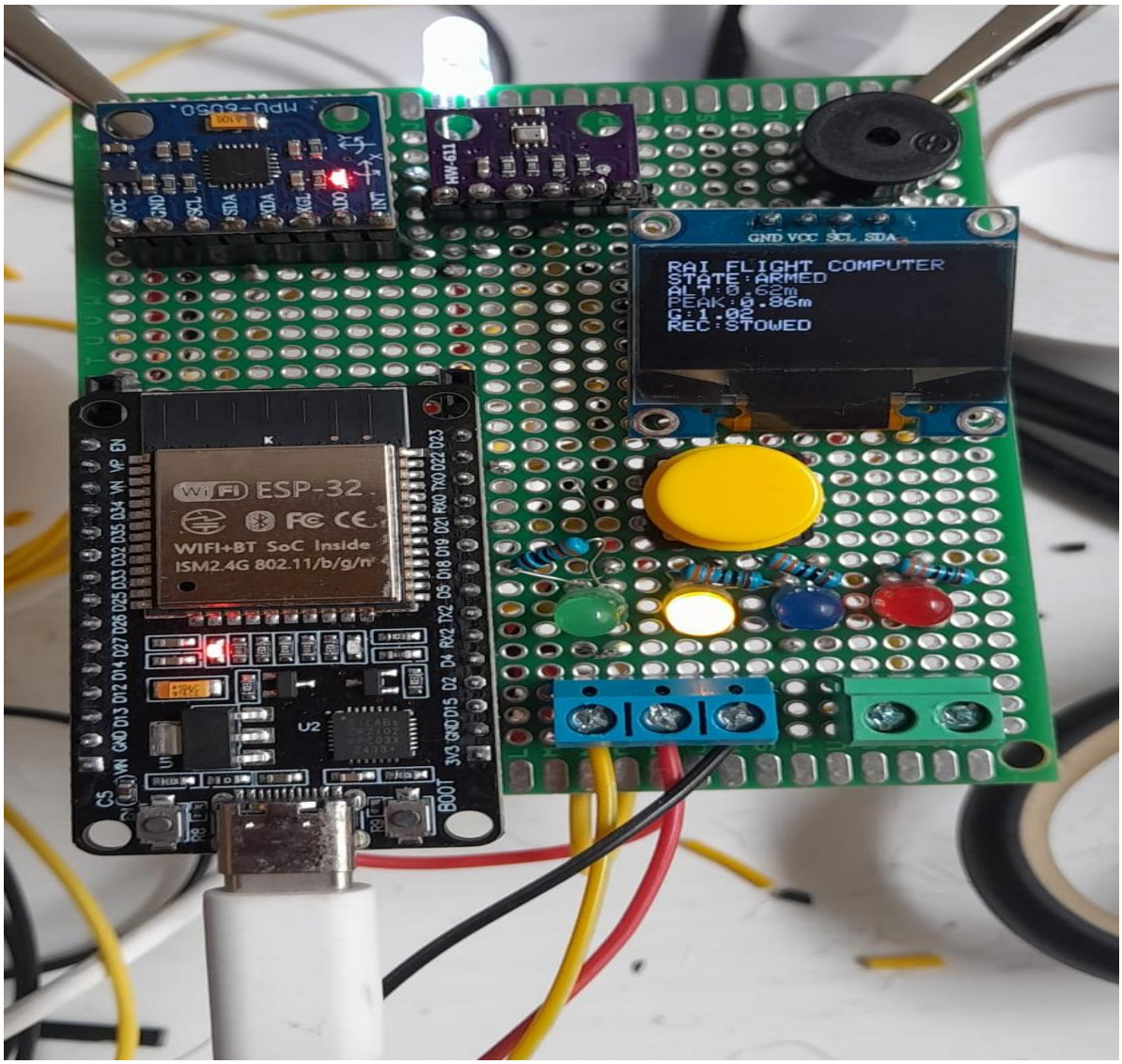
Following confirmed apogee detection, the state sequence continued:

ASCENT → APOGEE → DESCENT

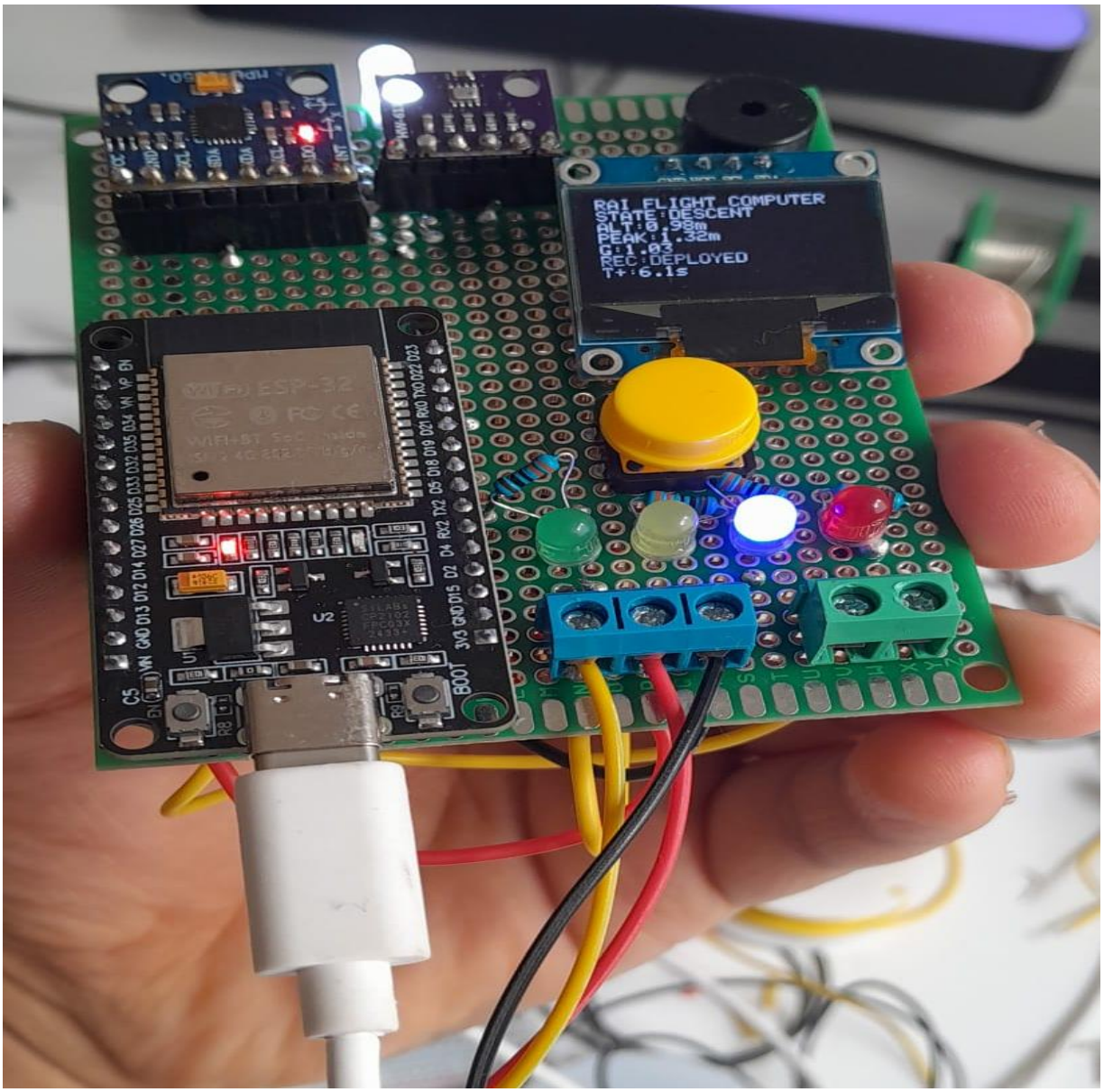
Landing confirmation subsequently resulted in:

DESCENT → LANDED

No direct transition from IDLE to ASCENT or from ASCENT directly to LANDED was intended by the implemented state logic.



*[**Figure 7-2**. FC_V1 in the ARMED state during bench verification prior to simulated launch-event detection.]*



*[**Figure 7-3**. FC_V1 operating in the DESCENT state during manual simulated-flight testing.]*

7.5 Apogee Detection and Recovery-Actuation Verification

Apogee verification focused on the interaction between peak-altitude tracking, altitude-decrease detection, confirmation logic and recovery actuation.

During ASCENT, the firmware continuously maintained the highest confirmed relative-altitude measurement as the recorded peak altitude. When the current altitude decreased sufficiently below this value, the system evaluated the apogee confirmation conditions.

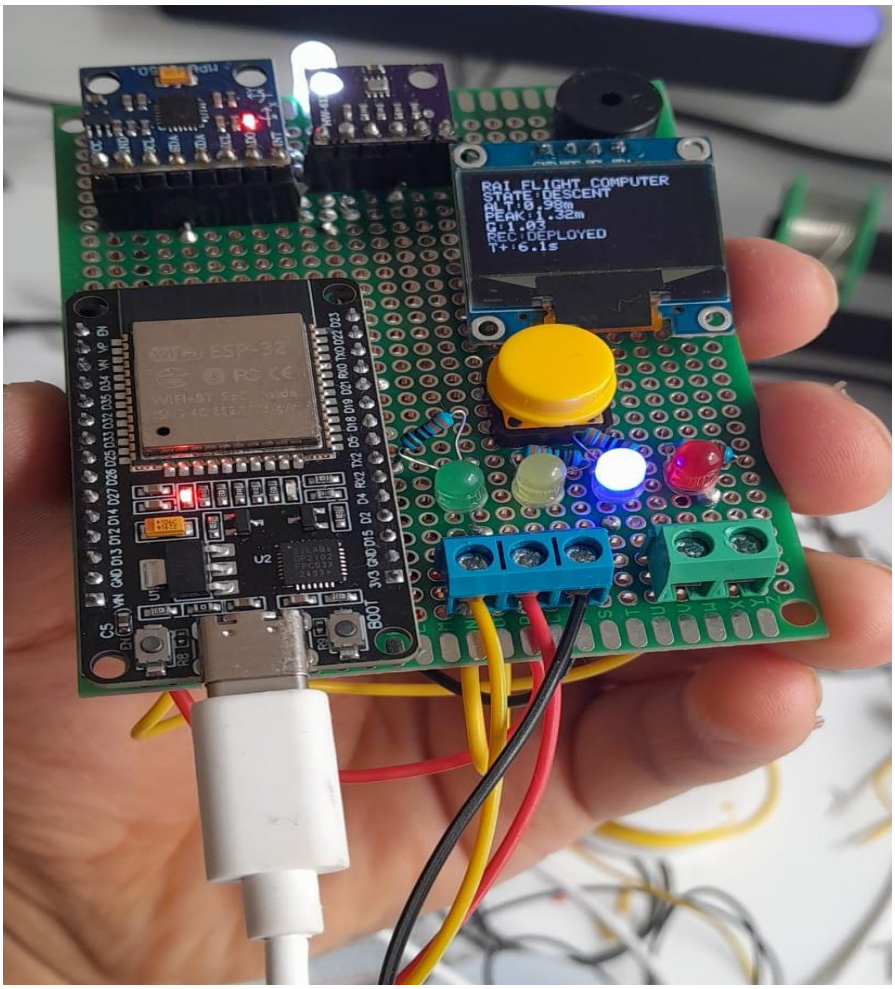
Recovery actuation was not commanded from an individual altitude sample. The servo output became eligible only after the firmware had confirmed the apogee event and entered the corresponding recovery sequence.

Successful operation was identified by the combination of:

Peak altitude recorded → Altitude decrease detected → APOGEE confirmed → Recovery command issued → Servo deployed → DESCENT entered

The OLED recovery-status field provided a direct indication of the actuator command state and allowed the transition from **STOWED** to **DEPLOYED** to be observed during testing.

This verified the complete software-to-hardware chain from sensor measurement and event detection to physical recovery-control output.



*[**Figure 7-4**. Recovery-output verification following confirmed apogee detection, with the OLED indicating the recovery*

actuator in the DEPLOYED condition.]

7.6 Landing Detection and Recovery-Beacon Verification

Following simulated descent, the prototype was returned to a stationary condition to evaluate landing detection.

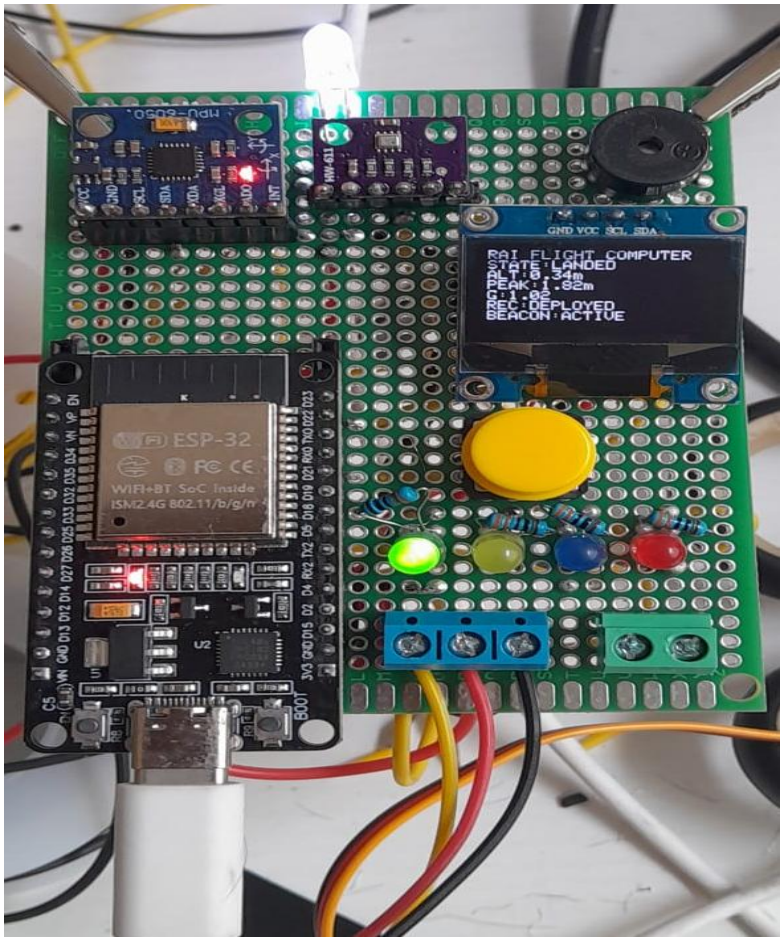
The firmware continued monitoring relative-altitude variation and acceleration magnitude during DESCENT. A landing event was not declared immediately when either measurement became temporarily stable. Instead, the implemented confirmation logic required the landing conditions to remain satisfied for the configured duration or count.

After confirmation, the state machine transitioned:

DESCENT → LANDED

Entry into LANDED activated the defined post-landing indication behaviour. The OLED displayed the final state and recovery condition, while the status indicators and audible beacon provided local recovery assistance.

Successful landing detection therefore verified the final portion of the implemented flight-state sequence and demonstrated that FC_V1 could progress from initial arming through simulated flight-event detection to post-landing recovery indication.



*[**Figure 7-5**. FC_V1 following confirmed simulated landing, showing the LANDED state, deployed recovery status and active recovery beacon.]*

7.7 Repeated Test Results

Table 7-1. Integrated FC_V1 Bench-Test Results

Test	Arming	Launch Detection	Ascent	Apogee Detection	Recovery Actuation	Descent	Landing Detection	Beacon	Overall
Test 1	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Test 2	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Test 3	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Test 4	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Test 5	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS

Five repeated integrated bench-test sequences were conducted using the final FC_V1 hardware and firmware configuration. The repeated trials were intended to evaluate the reproducibility of the implemented flight-state sequence under manually generated simulated-flight conditions.

The results demonstrate repeatable functional operation of the integrated prototype under the tested bench conditions. They should not, however, be interpreted as statistical reliability data or evidence of flight qualification because the test environment did not reproduce actual launch acceleration, vibration, aerodynamic loading, temperature variation or atmospheric flight conditions.

7.8 Test Limitations and Verification Status

The completed bench-test programme demonstrates that the FC_V1 prototype can execute its intended sensing, flight-state detection, recovery-command and post-landing indication functions under controlled simulated conditions.

However, several limitations remain.

The simulated altitude and acceleration profiles generated during manual testing do not reproduce the full dynamic environment expected during actual rocket flight. The prototype has not been subjected to controlled vibration testing, shock testing, thermal cycling, electromagnetic-compatibility testing or representative aerodynamic flight conditions.

The BMP280 altitude measurements remain susceptible to environmental pressure variations, while the MPU6050 measurements obtained during manual movement cannot fully represent the acceleration profile of a powered launch.

Similarly, although the recovery servo output has been functionally demonstrated, FC_V1 has not deployed an actual parachute or flight-recovery mechanism under representative aerodynamic loading.

Consequently, the verification status of FC_V1 is classified in this report as:

Integrated Functional Prototype — Bench Verified

rather than:

Flight Qualified

The completed tests establish a functional baseline for subsequent development and provide evidence that the fundamental sensing, state-machine and recovery-control architecture operates as intended under the tested conditions.

8. Engineering Assessment and Limitations

8.1 Overall Engineering Assessment

FC_V1 successfully demonstrates the development of an integrated embedded flight-event detection and recovery-control prototype using low-cost, commercially available electronic components.

The completed system combines barometric and inertial sensing, embedded processing, flight-state logic, user interaction, visual and audible status indication, and recovery-actuator control within a single functional hardware assembly.

Bench testing demonstrated that the implemented architecture can execute the intended sequence:

IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

and coordinate the associated recovery and post-landing functions under controlled simulated conditions.

From an engineering-development perspective, the principal achievement of FC_V1 is not individual sensor measurement or servo control in isolation. Rather, it is the successful integration of multiple hardware and software subsystems into a state-dependent embedded system in which sensor measurements influence event detection and subsequent physical outputs.

The prototype therefore provides a practical baseline for evaluating flight-computer architecture, embedded state-machine implementation, sensor integration and recovery-control logic before development of a more compact and flight-oriented successor.

However, FC_V1 remains an experimental engineering prototype. Its architecture, construction method, sensor configuration and verification level impose significant limitations that prevent the system from being considered flight-qualified hardware.

8.2 Hardware Limitations

The FC_V1 hardware was developed primarily for accessibility, modification and functional verification rather than minimum mass, minimum volume or environmental robustness.

The ESP32 development board provides sufficient processing capability for the implemented prototype functions, but the development-board format introduces unnecessary physical size and includes circuitry not specifically required for a dedicated flight computer.

Similarly, the perforated circuit-board construction and manually implemented point-to-point wiring provide a practical intermediate stage between breadboard development and a custom printed circuit board, but they are not optimal for a vibration-intensive flight environment.

The principal hardware limitations include:

- relatively large physical dimensions;
- comparatively high mass for a small flight-computer system;
- manually routed point-to-point wiring;
- limited mechanical strain relief;
- exposed electronic components;
- absence of a dedicated enclosure;
- non-locking module connections;
- development-oriented USB power/programming interface;
- lack of dedicated onboard data-storage hardware;
- no redundant sensing or processing architecture.

The modular socket arrangement remains useful during prototype development because individual components can be removed or replaced. However, the same arrangement may introduce additional mechanical connection points susceptible to vibration or intermittent contact.

A future hardware revision should therefore replace the perfboard architecture with a purpose-designed PCB and mechanically secured electrical interfaces.

Feature	FC_V1 Implementation	Engineering Assessment
Processing	ESP32 development board	Suitable for prototype development; oversized for dedicated flight hardware
Barometric sensing	BMP280	Suitable for functional altitude testing; requires flight validation
Inertial sensing	MPU6050	Suitable for prototype motion sensing; limited for higher-performance applications
Construction	Perfboard / point-to-point	Appropriate for bench prototype; poor long-term vibration robustness
Display	OLED	Valuable for development and diagnostics; not essential for flight hardware
Recovery output	Servo interface	Functional actuation demonstration; actual recovery mechanism not validated
Connections	Headers / screw terminals	Convenient for development; mechanical retention requires improvement
Data logging	Not implemented	Major limitation for post-flight analysis
Enclosure	None	Environmental and mechanical protection required

8.3 Sensor and Measurement Limitations

FC_V1 derives relative altitude from BMP280 barometric-pressure measurements. Although suitable for demonstrating altitude-based event detection, barometric altitude is influenced by atmospheric-pressure variation, local airflow and sensor noise.

The use of relative altitude established from an initialization reference reduces dependence on absolute elevation but does not eliminate transient pressure disturbances.

The MPU6050 provides the inertial measurements required by the implemented launch- and landing-detection logic. However, FC_V1 does not employ a high-performance inertial navigation architecture, redundant IMUs or advanced sensor-fusion algorithms.

The prototype therefore uses the available measurements primarily for **flight-event detection rather than navigation or attitude estimation**.

This distinction is important. FC_V1 should not be interpreted as providing full inertial navigation, vehicle guidance or closed-loop flight control.

The current sensor configuration is sufficient for investigating the implemented state-machine concept, but future development should evaluate improved sensor filtering, calibration, higher-performance inertial sensors and multi-sensor fusion.

8.4 Firmware and Algorithm Limitations

The FC_V1 firmware implements deterministic threshold- and confirmation-based flight-event detection.

This approach provides several advantages during early prototype development: the logic remains understandable, computationally inexpensive and relatively easy to observe during bench testing.

However, fixed detection thresholds may not remain optimal across vehicles with substantially different acceleration profiles, flight durations, expected apogees or recovery characteristics.

The implemented confirmation logic reduces sensitivity to isolated measurement fluctuations but does not completely eliminate the possibility of false or delayed event detection under flight conditions that differ significantly from the bench-test environment.

FC_V1 also lacks several capabilities commonly desirable in more advanced flight-computer architectures, including persistent onboard flight-data logging, configurable mission profiles, comprehensive fault detection, sensor redundancy and advanced state estimation.

Consequently, the current firmware should be considered a functional implementation of the FC_V1 experimental architecture rather than a generalized flight-computer software platform.

8.5 Recovery-System Limitations

The recovery subsystem represents one of the most important boundaries of the FC_V1 validation programme.

The prototype successfully demonstrates the electronic chain:

Sensor Measurement → Event Detection → Apogee Confirmation → Recovery Command → Servo Actuation

However, the servo output has not been validated with an actual parachute deployment mechanism under representative flight loading.

Therefore, successful servo movement during bench testing does **not** establish successful parachute deployment capability.

Actual recovery-system validation would additionally require consideration of mechanical retention, deployment energy, aerodynamic loading, parachute packing, deployment timing, actuator reliability and failure modes.

FC_V1 consequently demonstrates **recovery-control actuation**, not a flight-qualified recovery system.

8.6 Reliability and Environmental Limitations

The prototype has not undergone formal environmental qualification.

No controlled testing has been performed for:

Vibration • Shock • Temperature • Humidity • Electromagnetic Compatibility • Sustained Acceleration • Reduced Pressure

The electrical interconnections and removable modules were developed for laboratory accessibility and may behave differently when exposed to launch vibration or shock.

Similarly, power integrity has been evaluated only within the prototype's bench operating configuration. A dedicated flight power architecture, battery-management strategy and comprehensive power-failure analysis have not yet been implemented.

The repeated successful bench tests reported in Section 7 demonstrate functional repeatability under the evaluated conditions, but they do not provide sufficient evidence to calculate system reliability or probability of mission success.

8.7 FC_V1 Technology Status

Based on the completed development and verification activities, FC_V1 can reasonably be classified as:

Integrated Functional Prototype — Bench Verified

The project demonstrates:

Sensor Acquisition → Embedded Processing → Flight-State Estimation → Event Detection → Recovery Command → Physical Actuation → Post-Landing Indication

within a single integrated prototype.

It does not demonstrate flight qualification, production readiness or environmental qualification.

FC_V1 therefore serves primarily as a **proof-of-function engineering platform and development baseline** for the next flight-computer revision.

Table 8-2. FC_V1 Verification Boundary

Capability	**Status**
Individual sensor operation	Verified
Integrated sensor acquisition	Verified
Flight-state machine	Bench Verified
Launch detection	Simulated / Bench Verified
Apogee detection	Simulated / Bench Verified
Recovery command	Bench Verified
Servo actuation	Bench Verified
Landing detection	Simulated / Bench Verified
Recovery beacon	Bench Verified
Repeated integrated sequence	Bench Verified
Actual rocket flight	Not Tested
Actual parachute deployment	Not Tested
Vibration qualification	Not Tested
Thermal qualification	Not Tested

Flight qualification	Not Achieved

9. Future Development and FC_V2 Roadmap

The development and bench verification of FC_V1 identified several areas in which the architecture can be improved for future flight-oriented applications. FC_V1 established the fundamental sensing, state-machine and recovery-control architecture; however, the limitations identified in Section 8 provide clear engineering requirements for the next development stage.

The proposed FC_V2 architecture should therefore focus on improved mechanical robustness, reduced physical size, onboard data recording, improved sensing and filtering, more robust power distribution and greater suitability for integration into an experimental flight vehicle.

The objective of FC_V2 should not simply be to reproduce FC_V1 on a smaller board. Instead, the next revision should convert the functional architecture demonstrated by FC_V1 into a more integrated and testable flight-computer platform.

9.1 Dedicated PCB Development

One of the principal hardware improvements proposed for FC_V2 is the replacement of the manually wired perfboard architecture with a purpose-designed printed circuit board.

The PCB should integrate the principal electrical interfaces required by the flight computer while reducing wiring complexity and the number of manually implemented electrical connections.

A dedicated PCB would provide several advantages over FC_V1, including:

- reduced physical dimensions;
- reduced wiring complexity;
- improved electrical consistency;
- improved mechanical robustness;
- more controlled power and ground routing;
- easier inspection and assembly;
- defined mounting locations;
- improved repeatability between hardware units.

The PCB design should also provide clearly defined interfaces for sensors, power input, programming/debugging, recovery outputs and any external peripherals required by the selected vehicle.

Development should progress through schematic capture, electrical-rule checking, PCB layout, design-rule checking, fabrication and controlled bench verification before flight integration is considered.

9.2 Onboard Data Logging

A major functional improvement for FC_V2 should be persistent onboard data recording.

FC_V1 provides real-time diagnostic information through the OLED and serial interface, but does not preserve a complete flight dataset after power removal. This substantially limits quantitative post-test and post-flight analysis.

FC_V2 should therefore incorporate an onboard logging system capable of recording selected parameters such as:

- timestamp;
- flight state;
- relative altitude;
- barometric pressure;
- acceleration;
- angular-rate measurements;
- recorded peak altitude;
- detected flight events;
- recovery-command state;
- system status and fault information.

A removable storage device such as a microSD card or suitable onboard non-volatile memory could be evaluated depending on the required logging rate, reliability and physical constraints.

Recorded datasets would allow sensor behaviour, event timing and state transitions to be analysed after testing and would provide objective evidence for subsequent algorithm development.

9.3 Improved Sensor Processing and State Estimation

FC_V1 demonstrates flight-event detection using barometric and inertial measurements with threshold- and confirmation-based logic.

Future development should investigate improved filtering and multi-sensor processing to increase resistance to transient measurements and environmental disturbances.

Potential improvements include digital filtering of barometric and inertial measurements, sensor calibration routines, improved bias handling and correlation between independent measurements when determining flight events.

Where justified by future vehicle requirements, more advanced sensor-fusion methods may also be evaluated.

The objective should not necessarily be to introduce computational complexity for its own sake. Any additional filtering or estimation method should provide a measurable improvement in event-detection reliability or flight-data quality.

9.4 Power-System Development

FC_V2 should incorporate a more clearly defined flight-power architecture than the development-oriented arrangement used for FC_V1.

The power system should provide stable supply conditions for the processing unit, sensors and recovery-control hardware while accounting for transient loads generated by external actuators.

Future development should evaluate:

- dedicated battery input;
- appropriate voltage regulation;
- power filtering and decoupling;
- reverse-polarity protection where appropriate;
- separation or isolation of high-current actuator loads where required;
- battery-voltage monitoring;
- brownout behaviour;
- secure electrical connectors.

Power-system verification should include controlled testing across the expected operating-voltage range and observation of system behaviour during recovery-actuator operation.

9.5 Mechanical Integration and Enclosure

FC_V1 was intentionally constructed as an accessible bench prototype and therefore does not incorporate a dedicated protective enclosure.

A future flight-oriented revision should include defined mechanical mounting and protection for the electronic assembly.

The mechanical design should consider:

- PCB mounting points;
- connector retention;
- strain relief;
- protection against accidental electrical contact;
- vibration transmission;
- sensor exposure requirements;
- access for programming and data retrieval;
- installation within the intended vehicle.

Particular attention should be given to the barometric sensor because pressure measurements require appropriate exposure to the surrounding static-pressure environment without excessive disturbance from vehicle airflow.

9.6 Recovery-System Development

The FC_V1 recovery output demonstrates software-controlled servo actuation following confirmed apogee detection. Future development must extend verification beyond electronic actuation to the complete recovery mechanism.

The next development stage should therefore include a representative mechanical deployment system and controlled ground testing of the complete recovery chain.

The verification sequence should progressively evaluate:

Flight-Event Detection → Recovery Command → Actuator Operation → Mechanical Release → Recovery-System Deployment

Testing should initially be performed under controlled ground conditions before any flight application is attempted.

Recovery-system development should also consider failure modes such as actuator failure, mechanical jamming, insufficient deployment energy, electrical disconnection and loss of system power.

9.7 Environmental and Flight Verification Roadmap

Progression from FC_V1 bench verification toward a flight-oriented system should occur incrementally.

A proposed verification sequence is:

Stage 1 — Functional Bench Testing
Individual subsystem and integrated functional verification.

Stage 2 — Extended Ground Testing
Long-duration operation, power testing, repeated event sequences and fault-condition testing.

Stage 3 — Mechanical and Vibration Evaluation
Assessment of electrical connections, mounting and structural integrity under representative mechanical disturbance.

Stage 4 — Integrated Recovery Ground Testing
Verification of the complete physical recovery-actuation mechanism.

Stage 5 — Controlled Low-Risk Flight Demonstration
Operation aboard an appropriate experimental platform under defined test conditions.

Stage 6 — Post-Flight Analysis and Revision
Evaluation of recorded flight data, detected events, hardware condition and identified failure modes.

Successful completion of one stage should provide evidence supporting progression to the subsequent stage rather than assuming flight readiness from bench functionality alone.

9.8 Proposed FC_V2 Development Objectives

Table 9-1. Proposed FC_V2 Development Objectives

Area	FC_V1	Proposed FC_V2 Direction
Circuit architecture	Perfboard / point to point	Dedicated PCB
Processing	ESP32 development board	More integrated processing architecture
Altitude sensing	BMP280	Evaluate improved barometric sensing
Inertial sensing	MPU6050	Evaluate improved IMU
Data logging	Not implemented	Persistent onboard logging
Sensor processing	Threshold-based	Improved filtering and sensor processing
Power architecture	Development-oriented	Dedicated flight-Power architecture
Connections	Headers / terminals	Mechanically secured interfaces
Mechanical protection	None	Dedicated enclosure
Recovery	Servo actuation demonstrated	Integrated recovery-mechanism testing
Verification	Bench verified	Ground/environmental testing followed by controlled flight testing
Post-flight analysis	Limited	Recorded sensor/event datasets

9.9 Development Philosophy

FC_V1 established that the fundamental flight-event detection and recovery-control concept can operate as an integrated embedded prototype.

FC_V2 should build upon this baseline by improving the aspects that FC_V1 intentionally left at prototype level rather than replacing the architecture without evidence.

The development approach should therefore remain incremental:

Design → Implement → Test → Record → Analyse → Improve

Each hardware or firmware modification should be associated with a defined engineering objective and subsequently verified against measurable test criteria.

This approach allows FC_V1 to serve not merely as an isolated prototype, but as the first hardware iteration in a continuing flight-computer development programme

10. Conclusion

10.1 Project Summary

The FC_V1 project developed and verified an integrated embedded flight-event detection and recovery-control prototype intended to establish a practical foundation for future flight-computer development.

The system combines an ESP32 processing platform with BMP280 barometric sensing, MPU6050 inertial sensing, an OLED diagnostic interface, visual and audible status indication, user input and a servo-based recovery-control output.

Development progressed incrementally from individual component preparation and subsystem testing through breadboard integration, perfboard construction, firmware development and integrated bench verification.

The completed prototype implements a state-based flight-event sequence:

IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

Sensor measurements are used by the firmware to evaluate flight-event conditions, maintain peak-altitude information, command recovery actuation following confirmed apogee and activate post-landing indication following confirmed landing.

10.2 Verification Outcome

Integrated bench testing demonstrated that FC_V1 could repeatedly execute the intended flight-state sequence under manually generated simulated conditions.

Five repeated integrated bench-test sequences successfully demonstrated:

- system initialization and sensor acquisition;
- user-controlled arming;
- simulated launch detection;
- transition into ASCENT;
- peak-altitude tracking;
- apogee detection;
- recovery-command generation;
- servo actuation;
- transition through DESCENT;
- landing detection; and
- post-landing visual and audible indication.

These results establish FC_V1 as an:

Integrated Functional Prototype — Bench Verified

The verification results do not establish flight qualification, environmental qualification or operational reliability under actual rocket-flight conditions.

10.3 Engineering Outcome

The principal engineering outcome of FC_V1 is the successful integration of sensing, embedded processing, flight-event logic, user interaction and physical recovery-control output into a single functioning prototype.

The project therefore extends beyond individual sensor demonstrations. It establishes an end-to-end functional chain:

Physical Measurement → Sensor Acquisition → Embedded Processing → State Estimation → Event Detection → Recovery Command → Physical Actuation → Post-Landing Indication

The development process also identified practical limitations associated with prototype hardware, including point-to-point wiring, physical size, lack of onboard data logging, development-oriented power architecture, limited mechanical protection and the absence of environmental and flight verification.

These limitations provide defined engineering objectives rather than unresolved project failures and form the basis of the proposed FC_V2 development programme.

10.4 Final Conclusion

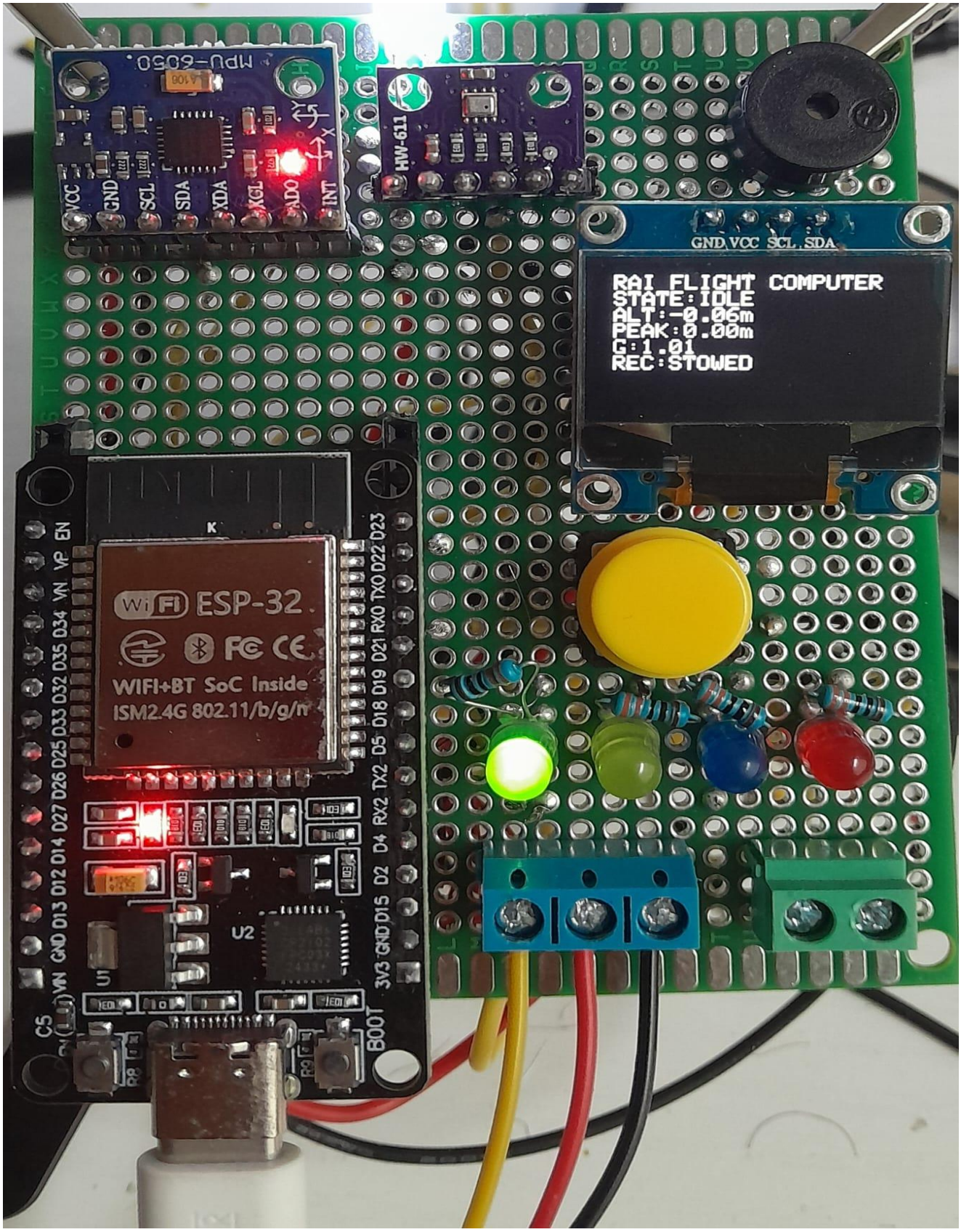
FC_V1 achieved its primary objective of demonstrating a functional embedded architecture capable of sensing representative flight conditions, executing deterministic flight-state logic and coordinating recovery-related outputs under controlled bench-test conditions.

The prototype should therefore be regarded as a **proof-of-function engineering platform** rather than a flight-ready avionics system.

Its value lies in establishing and experimentally verifying the fundamental architecture upon which subsequent hardware and firmware revisions can be developed.

Future work will focus on converting this development-oriented prototype into a more compact, mechanically robust and data-capable system through dedicated PCB development, improved sensing and signal processing, onboard data logging, dedicated power architecture, integrated recovery-system testing and progressively more representative environmental and flight verification.

FC_V1 therefore represents the first experimentally verified hardware iteration in the continuing development of the RAI flight-computer architecture.



***[Figure 10-1**. Completed FC_V1 integrated functional prototype following hardware integration, firmware implementation and bench verification.]*

11. References

[1] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, Shanghai, China.

[2] Bosch Sensortec, *BMP280 Digital Pressure Sensor Datasheet*, Bosch Sensortec GmbH.

[3] InvenSense, *MPU-6000 and MPU-6050 Product Specification*, InvenSense Inc.

[4] Arduino, *Arduino IDE Documentation*, Arduino.

[5] Espressif Systems, *Arduino-ESP32 Documentation*, Espressif Systems.

[6] Solomon Systech, *SSD1306 OLED/PLED Segment/Common Driver with Controller Datasheet*, Solomon Systech Ltd.

[7] TowerPro, *SG90 Micro Servo Technical Specifications*, TowerPro.

12. Appendices

Appendix A — FC_V1 Hardware Configuration and Pin Mapping

A.1 Hardware Configuration

The final FC_V1 prototype integrates the ESP32 processing platform with barometric and inertial sensing, local status indication, user input and recovery-control interfaces.

The principal electronic subsystems are:

Component	**Function**
ESP32 development board	Main processing and firmware execution
BMP280	Barometric-pressure and relative-altitude measurement
MPU6050	Inertial acceleration and angular-rate measurement
SSD1306 OLED display	Local flight-state and diagnostic information
SG90 servo interface	Recovery-actuator demonstration
Push button	User ARM/DISARM/RESET input
Status LEDs	Visual state/status indication
Buzzer	Audible post-landing recovery indication
Screw terminals	External electrical connections

A.2 Electrical Pin Mapping

The principal ESP32 GPIO assignments implemented in FC_V1 are listed below.

Function	ESP32 Connection
I²C SDA	GPIO 21
I²C SCL	GPIO 22
Recovery servo signal	GPIO 18
Buzzer	GPIO 25

A.3 I²C Sensor Architecture

The BMP280, MPU6050 and OLED display communicate with the ESP32 through the shared I²C bus.

The configuration allows multiple addressed peripherals to operate using the common SDA and SCL communication lines while retaining independent device addressing.

The shared communication architecture reduces the number of required ESP32 GPIO connections and provided a practical interface for the FC_V1 prototype.

Appendix B — Bill of Materials

B.1 Prototype Bill of Materials

FC_V1 was constructed primarily from commercially available development modules and standard electronic prototyping components. The use of modular hardware enabled individual subsystem testing, replacement and modification during development.

The principal components incorporated into the final prototype are summarized below.

Table B-1. FC_V1 Prototype Bill of Materials

Component	Qty.	Application in FC_V1
ESP32 development board	1	Main processing and firmware execution
BMP280 sensor module	1	Barometric-pressure and relative-altitude measurement
MPU6050 sensor module	1	Acceleration and angular-rate measurement
0.96-inch SSD1306 OLED	1	Flight-state and diagnostic display
SG90 micro servo	1	Recovery-actuator demonstration
Momentary push button	1	User input for system control
Status LEDs	4	Visual state and status indication
Piezo/active buzzer	1	Audible recovery indication
Perfboard	1	Mechanical and electrical integration platform
Female header sockets	As required	Removable mounting of principal modules
Screw-terminal blocks	As required	External electrical connections
Resistors	As required	LED current limiting and interface support
Hook-up wire	As required	Point-to-point electrical interconnection
Header pins/connectors	As required	Module and peripheral interfaces

B.2 Prototype Construction Materials

In addition to the principal electronic components, development required supporting prototyping materials and equipment including solder, flux, jumper wires, header pins and electrical connection hardware.

These items supported module preparation, breadboard verification and subsequent transfer to the permanent perfboard configuration.

The resulting construction approach was appropriate for a first integrated functional prototype because components could be installed, removed and modified without requiring fabrication of a dedicated printed circuit board.

However, these materials and construction techniques are development-oriented and are not proposed as the final implementation method for FC_V2.

B.3 BOM Classification

The FC_V1 bill of materials should therefore be regarded as a **prototype-level engineering BOM** rather than a production configuration.

Component selection was primarily driven by:

Availability → Development accessibility → Functional suitability → Ease of integration → Cost

rather than optimization for:

Mass → Volume → Power consumption → Environmental robustness → Flight qualification → Production repeatability

This distinction is important because the FC_V1 BOM documents the hardware required to demonstrate the architecture, while the component selection for FC_V2 should be reconsidered against the requirements of the intended flight vehicle.

Appendix C — Firmware Architecture and State Definitions

C.1 Firmware Architecture

The FC_V1 firmware provides the software layer connecting sensor acquisition, flight-event detection, state management, user interaction and recovery-control output.

At a functional level, firmware operation can be represented as:

Initialization → Sensor Acquisition → Measurement Processing → Flight-State Evaluation → Event Detection → Output Command → Status Indication

During operation, the ESP32 repeatedly acquires data from the BMP280 and MPU6050 and evaluates the resulting measurements against the conditions associated with the current flight state.

State transitions occur only when the corresponding event-detection and confirmation conditions are satisfied. This prevents the recovery sequence from being controlled directly by an isolated sensor measurement.

The OLED, LEDs and buzzer provide diagnostic and status information, while the servo interface provides the physical recovery-control output.

C.2 Flight-State Definitions

Table C-1. FC_V1 Flight-State Definitions

State	Principal Function
IDLE	System initialized; sensors monitored while flight-event transitions remain inhibited
ARMED	System prepared for launch-event detection
ASCENT	Launch confirmed; altitude monitored and peak altitude continuously updated
APOGEE	Apogee criteria confirmed; recovery sequence initiated
DESCENT	Recovery output active and system monitors conditions required for landing detection
LANDED	Landing confirmed; post-landing visual and audible recovery indication active

C.3 State-Transition Sequence

The implemented nominal state progression is:

IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

Each transition represents a defined change in system behaviour rather than merely a change in displayed status.

The state-machine architecture ensures that flight events are evaluated according to their expected sequence. For example, apogee evaluation occurs following confirmed ascent rather than during IDLE operation, and landing detection is evaluated after entry into the descent phase.

This sequential architecture reduces the possibility of logically invalid transitions during normal operation.

C.4 Recovery-Control Logic

Recovery actuation is associated with confirmed apogee detection rather than a single instantaneous altitude measurement.

The functional chain is:

Sensor Measurement → Peak-Altitude Tracking → Altitude-Decrease Detection → Apogee Confirmation → Recovery Command → Servo Actuation → DESCENT

This approach provides a clear separation between measurement acquisition, event interpretation and physical actuation.

FC_V1 demonstrates this control chain during bench testing; it does **not** establish successful deployment of an actual parachute or flight-recovery mechanism.

Appendix D — Bench-Test Records

D.1 Integrated Bench-Test Programme

Following completion of hardware integration and firmware development, FC_V1 was subjected to repeated manual bench-test sequences to evaluate the reproducibility of the implemented flight-state and recovery-control logic.

Each test was performed using the completed perfboard prototype and the integrated FC_V1 firmware configuration. Representative launch, ascent, apogee, descent and landing conditions were generated manually through controlled movement and sensor-input variation.

The purpose of these tests was not to reproduce the complete physical environment of rocket flight, but to determine whether the integrated prototype could repeatedly execute the intended logical sequence under controlled simulated conditions.

The nominal sequence evaluated during each test was:

IDLE → ARMED → ASCENT → APOGEE → DESCENT → LANDED

Successful completion required correct state progression together with the expected recovery-actuator, visual-status and post-landing outputs.

D.2 Integrated Test Results

Table D-1. Repeated FC_V1 Integrated Bench-Test Record

Function / Event	Test 1	Test 2	Test 3	Test 4	Test 5
Initialization	PASS	PASS	PASS	PASS	PASS
Arming	PASS	PASS	PASS	PASS	PASS
Launch detection	PASS	PASS	PASS	PASS	PASS
ASCENT transition	PASS	PASS	PASS	PASS	PASS
Peak-altitude tracking	PASS	PASS	PASS	PASS	PASS
Apogee detection	PASS	PASS	PASS	PASS	PASS
Recovery command	PASS	PASS	PASS	PASS	PASS
Servo actuation	PASS	PASS	PASS	PASS	PASS
DESCENT transition	PASS	PASS	PASS	PASS	PASS
Landing detection	PASS	PASS	PASS	PASS	PASS
LANDED transition	PASS	PASS	PASS	PASS	PASS
Recovery beacon/status	PASS	PASS	PASS	PASS	PASS
Overall result	**PASS**	**PASS**	**PASS**	**PASS**	**PASS**

Five repeated integrated test sequences were completed successfully using the final FC_V1 hardware and firmware configuration.

No unintended state transition or failure to complete the nominal sequence was observed during the recorded test series. The repeated results therefore provide evidence of functional repeatability under the specific controlled bench-test conditions employed.

The results must not be interpreted as statistical reliability data or evidence of flight qualification. The number of trials was limited, and the tests did not reproduce launch vibration, aerodynamic loading, temperature variation, representative pressure dynamics or other environmental conditions associated with actual flight.

D.3 Functional Acceptance Criteria

The following conditions were used as the principal functional acceptance criteria for the integrated FC_V1 prototype:

1. Successful initialization of the ESP32 and connected peripherals.
2. Stable acquisition of BMP280 and MPU6050 measurements.
3. Correct operation of the OLED and status indicators.
4. User-controlled transition from IDLE to ARMED.
5. Launch detection only after the required simulated launch conditions.
6. Correct transition from ARMED to ASCENT.
7. Continuous peak-altitude tracking during simulated ascent.
8. Apogee declaration only following the implemented confirmation conditions.
9. Recovery command generated following confirmed apogee.
10. Observable recovery-servo actuation.
11. Correct transition to DESCENT.
12. Landing declaration only after the required confirmation conditions.
13. Correct transition to LANDED.
14. Activation of the defined post-landing visual and audible recovery indications.

A test sequence was classified as **PASS** only when the complete required sequence was executed without an unintended state transition or missing required output.

D.4 Verification Boundary

The bench-test record verifies the functional behaviour of the prototype only within the conditions under which testing was performed.

Verified

Sensor acquisition → State-machine execution → Simulated flight-event detection → Recovery command → Servo actuation → Landing detection → Post-landing indication

Not verified

Actual rocket flight → Actual parachute deployment → Aerodynamic loading → Launch vibration → Thermal environment → Electromagnetic compatibility → Flight reliability

Accordingly, the test record supports the classification:

Integrated Functional Prototype — Bench Verified

It does not support classification of FC_V1 as flight-qualified hardware.

Appendix E — Development Evidence

E.1 Prototype Development Progression

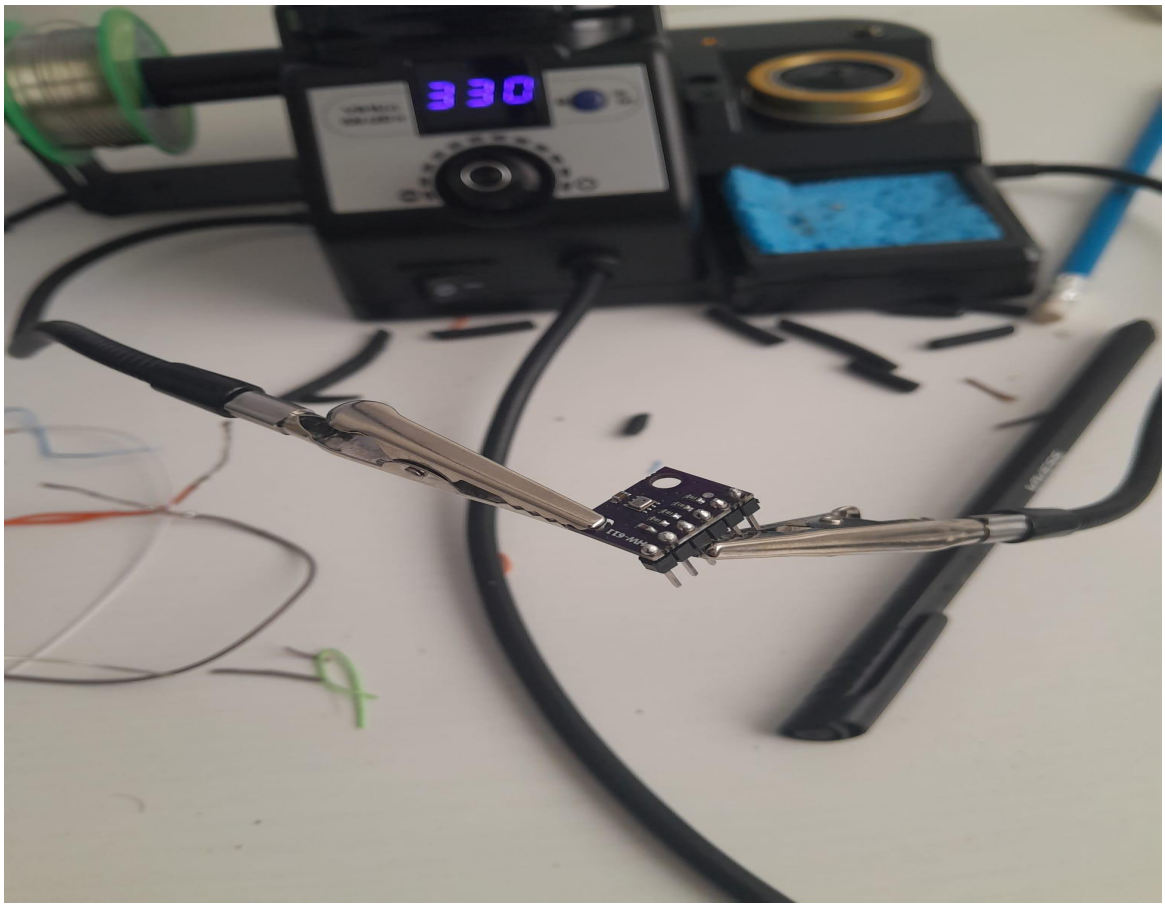
FC_V1 was developed through an incremental hardware-development process in which individual components were first prepared and verified before integration into progressively more complete configurations.

The principal development stages were:

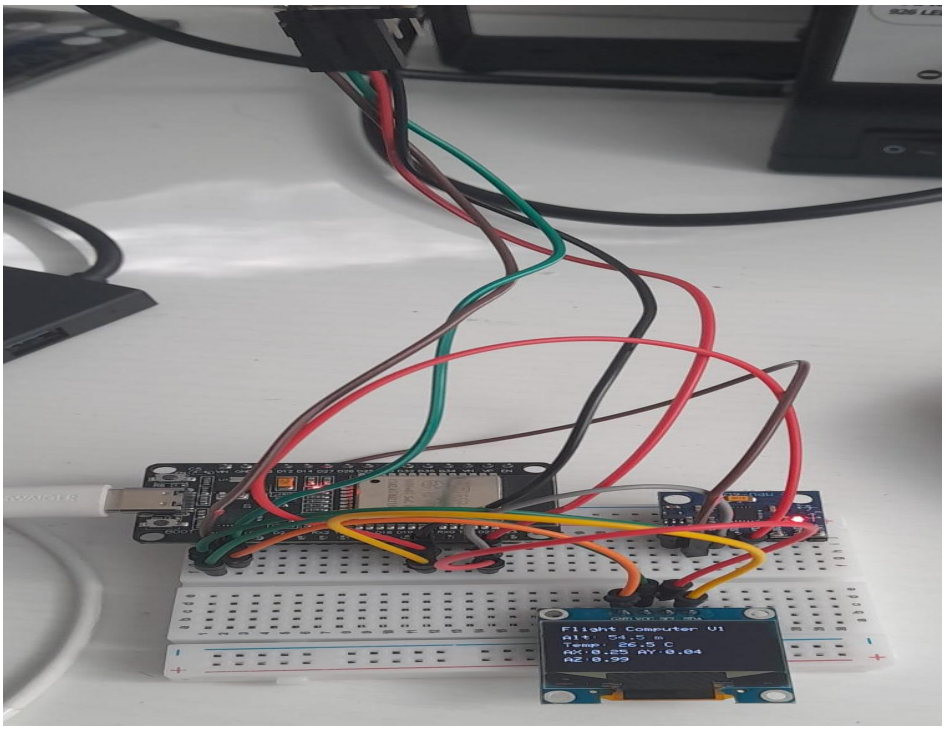
Component Preparation → Individual Module Testing → Breadboard Integration → Combined Subsystem Verification → Perfboard Construction → Point-to-Point Wiring → Integrated Firmware Testing → Repeated Bench Verification

This development sequence allowed hardware, wiring and firmware faults to be identified at subsystem level before evaluation of the complete prototype.

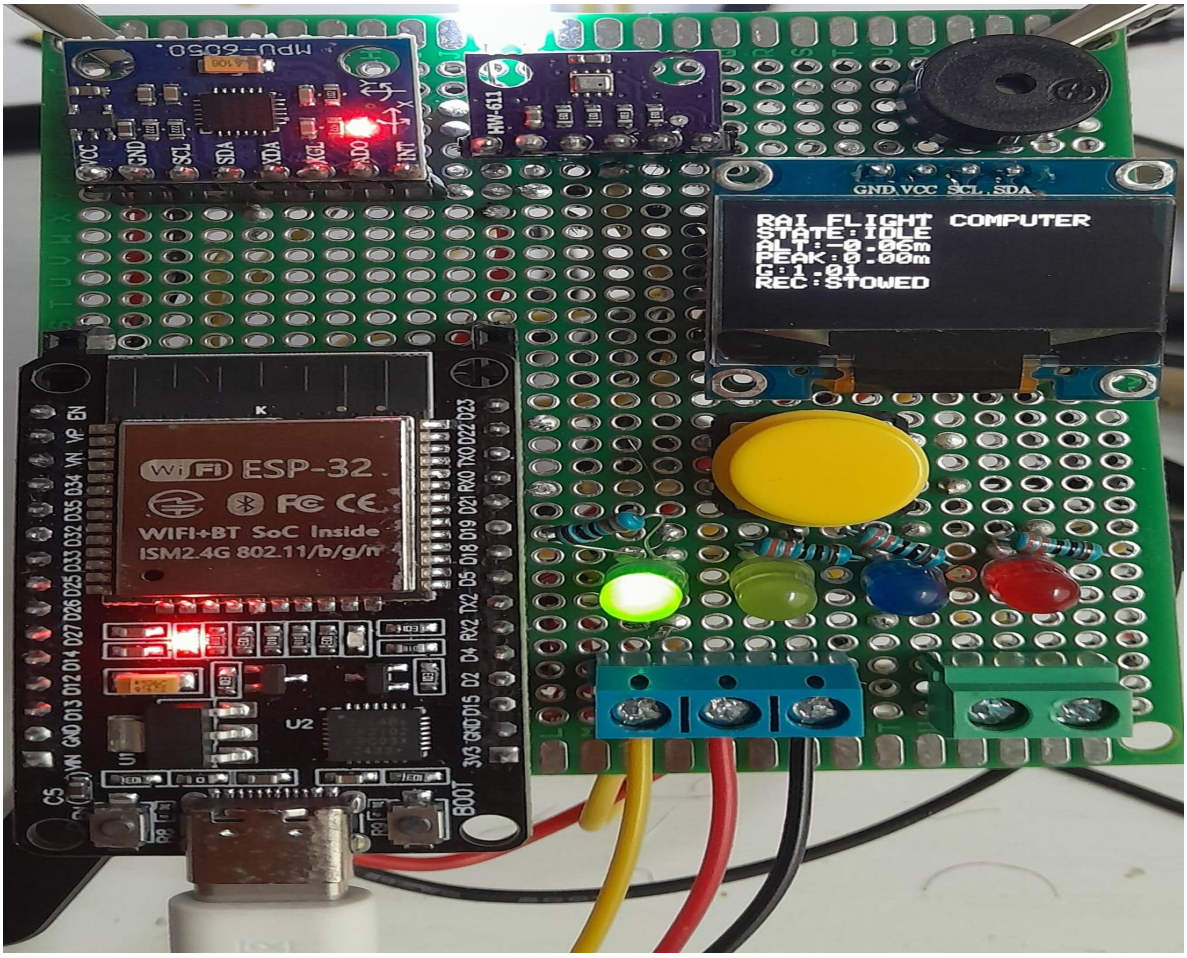
E.2 Representative Development Evidence



***[Figure E-1**. Representative component-preparation stage during FC_V1 development, showing header-pin soldering prior to subsystem integration.]*



*[**Figure E-2**. FC_V1 breadboard development configuration used for combined sensor, processing and display verification prior to permanent hardware integration.]*



*[**Figure E-3**. Completed FC_V1 integrated functional prototype following perfboard construction, firmware integration and bench verification.]*

E.3 Development Record Status

The photographs presented in this appendix provide representative evidence of the physical development progression of FC_V1. Detailed subsystem behaviour, integration methodology and verification results are documented in the corresponding sections of the main report.

Together with the hardware configuration, firmware architecture and bench-test records, this development evidence documents the transition of FC_V1 from individual development modules to an integrated and repeatedly bench-verified prototype.